



TRIBHUVAN UNIVERSITY

ASIAN SCHOOL OF MANAGEMENT & TECHNOLOGY

Gongabu, Kathmandu, Nepal

A Project Report

On

“LetsTalk- Real Time Language Practice Platform”

SUBMITTED BY:

AASHISH RIJAL

KUSHAL BHATTA

KEDAR DHUNGANA

A Project Report Submitted in partial fulfillment of the requirement of Bachelor of
Science in Computer Science & Information Technology (BSc.CSIT) 6th

Semester of Tribhuvan University, Nepal

Bhadra, 2082

LetsTalk – Real-Time Language Practice Platform

A project report submitted for the partial fulfillment of the requirement for the degree
of Bachelor of Science in Computer science & Information Technology awarded by

Tribhuvan University.

Submitted By

AASHISH RIJAL

KUSHAL BHATTA

KEDAR DHUNGANA

Submitted To

Asian School of Management & Technology

Department of Computer science

Affiliated to Tribhuvan University

Gongabu, Kathmandu Nepal



Bhadra, 2082

Date: 20th Bhadra, 2082

SUPERVISOR’S RECOMMENDATION

It is my pleasure to recommend that a report on “LetsTalk – Real-Time Language Practice Platform” has been prepared under my supervision by Aashish Rijal, Kushal Bhatta, and Kedar Dhungana in partial fulfillment of the requirement of the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT). Their report is satisfactory to process for the future evaluation.

.....

Mr. Hiranya Bastakoti

Supervisor

Asian School of Management & technology

Date: 20th Bhadra, 2082

CERTIFICATE OF APPROVAL

This is to certify that this report has been read and recommended to the Department of Computer Science and Information Technology for acceptance of report entitled “LetsTalk – Real-Time Language Practice Platform” submitted by Aashish Rijal, Kushal Bhatta and Kedar Dhungana in partial fulfillment for the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT), Institute of Science and Technology, Tribhuvan University.

.....

Ms. Anil Lal Amatya

Principle

.....

Mr. Hiranya Bastakoti

Supervisor

.....

Mr. Surya Bam

Research Coordinator

.....

External Examiner

Tribhuvan University

ACKNOWLEDGEMENT

We would like to express our honest gratitude to all those who contributed to this project. Our sincere thanks go to our supervisor Mr. Hiranya Bastakoti and our coordinator Mr. Surya Bam, who provided us with precious guidance and support throughout the project. Their invaluable expertise, encouragement, and support have been a constant source of inspiration. We would also like to thank Asian School of management and technology College for providing us with the resources and support needed to complete this project. Finally, we would like to extend our gratitude to our families and friends for their support and encouragement throughout the completion of the project.

With respect,

AASHISH RIJAL

KUSHAL BHATTA

KEDAR DHUNGANA

ABSTRACT

The *LetsTalk* platform is an innovative full-stack web application designed to facilitate real-time language practice, with a primary focus on English speaking skills. Users can log in, create or join virtual rooms, and engage in live voice or video conversations with real participants across the globe. The system incorporates features such as friend requests, following, user profiles with ratings and reviews, and AI-powered assistance to provide personalized feedback. Built using the MERN stack, WebRTC, and Socket.io for real-time communication, LetsTalk aims to create a global community of learners and speakers. The project applies an Agile iterative methodology, ensuring continuous improvement of both frontend and backend components. By fostering cross-cultural interaction and enhancing language fluency, LetsTalk addresses the growing need for accessible and interactive language learning platforms.

Keywords: Language Practice, Real-Time Communication, WebRTC, MERN Stack, AI Feedback, Social Networking.

TABLE OF CONTENT

TITLE PAGE.....	ii
SUPERVISOR'S RECOMMENDATION.....	iii
CERTIFICATE OF APPROVAL.....	iv
ACKNOWLEDGEMENT.....	v
ABSTRACT.....	vi
TABLE OF CONTENT.....	vii
LIST OF ABBREVIATIONS.....	ix
LIST OF FIGURES.....	xi
LIST OF TABLES.....	xiii
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	2
1.3 Objectives.....	2
1.4 Scope and Limitation.....	3
1.5 Development Methodology.....	3
1.6 Report Organization.....	5
CHAPTER 2: LITERATURE REVIEW.....	6
2.1 Background Study.....	6
2.2 Literature Review.....	6
CHAPTER 3: SYSTEM ANALYSIS.....	9
3.1 Project Analysis.....	9
3.1.1 Requirement Analysis.....	9
3.1.2 Feasibility Study.....	16
CHAPTER 4: SYSTEM DESIGN.....	24
4.1 Design Overview.....	24
4.2 Algorithm Details.....	41
CHAPTER 5: IMPLEMENTATION AND TESTING.....	48
5.1 Implementation.....	48
5.2 Testing.....	56
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION.....	74
6.1 Conclusion.....	74
6.2 Future Recommendations.....	74
REFERENCES.....	76
APPENDICES.....	78

LIST OF ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DBMS	Database Management System
DRY	Do not Repeat Yourself
ERP	Enterprise Resource Planning
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ICT	Information and Communication Technology
JWT	JSON Web Token
KISS	Keep It Simple Stupid
MERN	MongoDB, Express.js, React, Node.js
MVC	Model-View-Controller
OTP	One-Time Password
PHP	Hypertext Preprocessor
POS	Point of Sale
RAM	Random Access Memory
REGEX	Regular Expression
REST API	Representational State Transfer API
SQL	Structured Query Language
SSL	Secure Sockets Layer
UML	Unified Modeling Language
WebRTC	Web Real-Time Communication
WS	WebSocket

LIST OF FIGURES

Figure 1.1 Incremental Delivery Model	16
Figure 3.1 Use Case Diagram of User	20
Figure 3.2 Use Case Diagram of Admin	21
Figure 4.1 System Architecture	28
Figure 4.2 Module Interaction Diagram	29
Figure 4.3 Entity-Relationship Diagram	30
Figure 4.4 Security Flow Diagram	32
Figure 5.1 Homepage Example Code	34
Figure 5.2 Room Creation Example Code	35
Figure 5.3 Real-time Chat Example Code	36
Figure 5.4 User Profile Example Code	37
Figure 5.5 Authentication Example Code	38
Figure 5.6 Friend Request & Follow Example Code	39
Figure 5.7 Rating & Review Example Code	40
Figure 5.8 Notification System Example Code	41
Figure 5.9 User Schema Code	42
Figure 5.10 Room Schema Code	43
Figure 5.11 Message Schema Code	44
Figure 5.12 Review Schema Code	45

LIST OF TABLES

Table 3.1	Use Case Scenario for User Operations	20
Table 3.2	Use Case Scenario for Admin Operations	21
Table 3.3	Feasibility Study Table	25
Table 4.1	User Schema Field Details	30
Table 4.2	Room Schema Field Details	31
Table 4.3	ChatMessage Schema Field Details	32
Table 4.4	Notification Schema Field Details	33
Table 5.1	Test Cases for User Registration & Login	50
Table 5.2	Test Cases for Room Creation & Joining	51
Table 5.3	Test Cases for Real-time Chat	52
Table 5.4	Test Cases for Friend Request & Following	53
Table 5.5	Test Cases for Rating & Review System	54
Table 5.6	Test Cases for Notifications	55
Table 5.7	Test Cases for AI Feedback Feature	56

CHAPTER 1

INTRODUCTION

1.1 Introduction

The **LetsTalk** platform is an innovative web-based application designed to facilitate real-time language practice, with a primary focus on improving English speaking skills. It allows users to connect with language learners and speakers across the globe, creating opportunities for live voice and video conversations in virtual rooms. The platform is designed to be intuitive and user-friendly, enabling participants to easily join sessions, interact with others, and enhance their language fluency through engaging, real-time communication.

For learners, LetsTalk offers features such as AI-powered feedback, personalized ratings and reviews, and activity tracking, allowing users to monitor their progress and identify areas for improvement. The platform also supports social interaction through friend requests, following, and participation in group discussions, fostering a collaborative and motivating learning environment.

From the administrative perspective, LetsTalk provides a robust backend system for managing users, rooms, messages, and notifications. Administrators can oversee active sessions, monitor user activity, and ensure smooth operation of the platform. The system leverages modern technologies such as the **MERN stack**, **WebRTC**, and **Socket.io** to provide reliable, scalable, and real-time communication capabilities.

One of the key advantages of LetsTalk is its emphasis on global connectivity and cross-cultural exchange, allowing users to practice languages in authentic conversational settings. The platform also ensures secure interactions, efficient management of sessions, and accurate delivery of notifications.

Moreover, LetsTalk is developed with future scalability in mind, enabling integration with additional features such as mobile support, multi-language practice, AI tutoring enhancements, or third-party educational tools. Users to improve their communication skills while connecting with a global community.

1.2 Problem Statement

The growing demand for language learning has highlighted several challenges for learners seeking practical speaking experience. Traditional methods, such as classroom lessons or textbook exercises, often fail to provide real-time interaction and practical conversation practice. Learners frequently struggle to find partners who are available for live discussions, and opportunities for authentic cross-cultural communication are limited.

Moreover, existing online language platforms often lack personalized feedback, social engagement, or mechanisms to track progress effectively. Users may not receive constructive guidance on pronunciation, fluency, or conversational skills, making it difficult to measure improvement over time. The absence of real-time communication and interactive features reduces learner motivation and engagement.

From the platform management perspective, administrators face challenges in monitoring active sessions, managing user interactions, and ensuring the smooth operation of the system in real-time. Ensuring secure communication, managing notifications, and maintaining accurate records of users' activities also add complexity to system management.

This gap between learners' expectations for interactive, engaging, and practical language practice, and the limitations of current solutions, creates the need for an innovative platform. **LetsTalk** aims to bridge this gap by providing a full-featured real-time language practice environment. The platform will enable live voice and video conversations, offer AI-powered feedback, foster social interaction, and provide administrators with tools to efficiently manage users, rooms, and activities. By addressing these challenges, LetsTalk seeks to enhance language fluency, learner engagement, and global connectivity.

1.3 Objectives

The main objectives of the LetsTalk platform are:

1. **Enable Real-Time Language Practice:** To provide a platform where users can engage in live voice and video conversations, improving their speaking and listening skills through practical interaction.
2. **Enhance Learning with AI Feedback:** To offer personalized guidance and feedback using AI, helping users track their progress, improve pronunciation, and increase fluency over time.
3. **Foster Social Interaction and Community Building:** To create a global community where users can make friends, follow other learners, participate in group discussions, and share learning experiences.
4. **Streamline Platform Management:** To provide administrators with tools to manage users, rooms, messages, and notifications efficiently, ensuring smooth real-time operation of the system.
5. **Support Data-Driven Insights:** To collect and analyze user activity and session data, helping improve the learning experience, optimize platform performance, and plan for future enhancements.

By addressing these objectives, **LetsTalk** aims to provide an engaging, interactive, and effective environment for language learners while supporting efficient management and continuous improvement of the platform.

1.4 Scope and Limitation

1.4.1 Scope

The **LetsTalk** platform is designed to serve as a comprehensive solution for real-time language practice and social learning. The system enables users to create or join virtual rooms, engage in live voice and video conversations, and participate in interactive text chats. Users can build profiles, track their learning progress, receive AI-powered feedback, and connect with other learners globally.

The scope includes features such as friend requests, following, ratings and reviews, notifications, and activity tracking, creating a social and collaborative learning environment. The platform is accessible via desktop and web browsers, ensuring learners can practice anytime and from anywhere.

Additionally, the system allows administrators to efficiently manage users, rooms, messages, and notifications, ensuring smooth real-time operation. Data analytics capabilities are incorporated to track user activity, session participation, and learning outcomes, enabling continuous improvement of the platform.

1.4.2 Limitation

The platform initially supports English language practice; other languages may be added in future updates. Internet connectivity is required for real-time communication; users with low bandwidth may experience limitations. Mobile application support is limited in the initial phase; the platform is primarily designed for desktop and web browsers. AI-powered feedback is based on available algorithms and may not capture all nuances of language fluency. The system depends on user engagement; its effectiveness is limited if active participation is low.

By defining this scope and acknowledging the limitations, **LetsTalk** ensures a focused and manageable project while providing a foundation for future expansion and improvement.

1.5 Development Methodology

The **LetsTalk** platform is developed using the **Agile Development Methodology**, emphasizing flexibility, collaboration, and iterative progress. The development process is organized into sprints, where each sprint focuses on delivering key functionalities while ensuring a user-centric design.

Iteration 1: The first phase establishes the core infrastructure, including setting up the development environment, defining database schemas, and creating wireframes for the user interface. A minimum viable product (MVP) is developed, covering basic functionalities such as user registration, login, and room creation.

Iteration 2: This iteration focuses on enhancing platform features, including room joining, live text chat, and participant management. The backend database structure is refined to handle real-time interactions efficiently, and preliminary AI-powered feedback mechanisms are introduced.

Iteration 3: The third iteration integrates live voice and video communication using **WebRTC** and **Socket.io**. User profile management, friend requests, following, and notification systems are implemented to improve engagement and social interaction.

Iteration 4: In this phase, the user interface and experience are optimized for better usability. Detailed testing and bug fixing are conducted to ensure smooth operation of real-time features, and feedback from initial users is incorporated to refine system components.

Iteration 5: The final iteration introduces advanced features, including AI-powered personalized feedback, session analytics, and progress tracking. The system is fully tested for performance, security, and scalability, ensuring a robust, user-friendly, and engaging platform for global language learners.

By following this iterative Agile methodology, **LetsTalk** ensures continuous improvement, rapid delivery of features, and adaptability to user feedback throughout the development process.

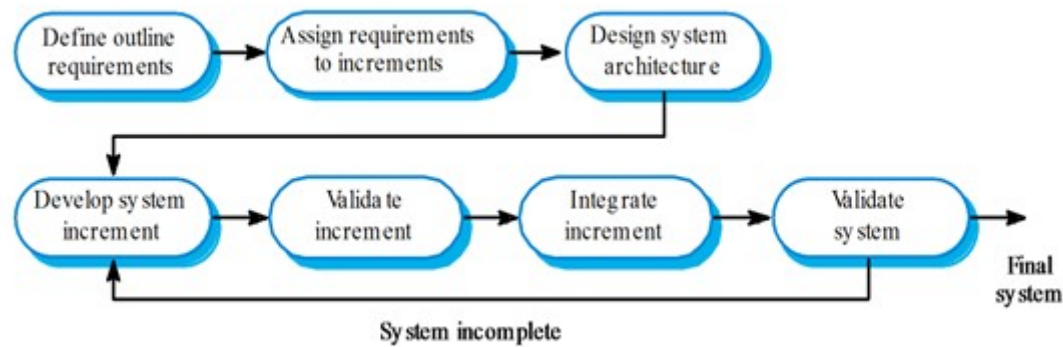


Figure 1.1 incremental Delivery

1.6 Report Organization

This project report is structured as follows:

Chapter 1: Provides an overview of the **LetsTalk** platform, including the problem statement, objectives, scope, limitations, and development methodology. This chapter sets the context for the entire project and explains the motivation behind creating a real-time language practice platform.

Chapter 2: Reviews existing literature and similar systems, discussing the current landscape of language learning platforms, real-time communication tools, and AI-assisted feedback systems, while identifying the gaps that **LetsTalk** addresses.

Chapter 3: Analyzes the proposed system, including system analysis, requirement specifications, and feasibility studies. It also presents use case diagrams for users and administrators.

Chapter 4: Discusses the system design, including architectural diagrams, module interaction, database schemas, and algorithmic details employed in the project.

Chapter 5: Outlines the tools, technologies, and frameworks used in the development of **LetsTalk**, such as the MERN stack, WebRTC, and Socket.io. It also includes testing methodologies, test cases, and sample outputs to ensure reliability and performance.

Chapter 6: Summarizes the outcomes of the project, evaluates the system against its objectives, discusses challenges faced, and provides recommendations for future improvements and enhancements.

By following this structure, the report presents a comprehensive view of the **LetsTalk** platform, from concept and design to implementation, testing, and future directions.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Language learning and practice have always been critical components of education and personal development. Traditional methods, such as classroom lessons, textbooks, and language labs, often fail to provide learners with adequate real-time interaction and practical conversational experience. Learners struggle to find suitable practice partners, and opportunities for cross-cultural communication are limited.

With the rise of the internet and digital technologies, online language learning platforms have emerged, offering courses, exercises, and virtual tutoring. However, many of these platforms focus primarily on theoretical knowledge or asynchronous learning, leaving a gap in real-time speaking and listening practice. Additionally, existing platforms often lack personalized feedback, social engagement, or tools to track user progress effectively.

The development of **LetsTalk** addresses these challenges by combining real-time voice and video communication, AI-powered feedback, and social networking features. By leveraging modern technologies such as the **MERN stack**, **WebRTC**, and **Socket.io**, LetsTalk provides an interactive and scalable platform that enables users to practice languages with real participants globally.

This background study highlights the evolution of language learning technologies, identifies gaps in existing solutions, and establishes the rationale for developing a dynamic, real-time language practice platform.

2.2 Literature Review

Language learning has evolved significantly with the advancement of technology. Researchers and developers have explored various platforms, tools, and methodologies to improve language acquisition and communication skills.

Several studies highlight the importance of real-time interaction for effective language learning. According to Warschauer (1997), computer-mediated communication enhances learner motivation, promotes authentic interaction, and improves speaking and listening skills. Platforms that enable live conversations with peers or tutors provide learners with the opportunity to practice pronunciation, fluency, and comprehension in realistic contexts.[1]

Existing language learning platforms, such as **Duolingo**, **HelloTalk**, and **Tandem**, offer mobile and web-based solutions that focus on vocabulary, grammar exercises, or asynchronous text and voice exchanges. While these platforms have made language learning more accessible, they often lack comprehensive real-time voice and video interaction with personalized feedback.[2]

Free4Talk is a platform that closely resembles LetsTalk, offering users the ability to join rooms and practice languages with real participants via voice or video. While Free4Talk enables basic real-time interaction, it lacks advanced features such as AI-powered feedback, detailed user progress tracking, personalized rating and review systems, and a robust social networking environment.[3]

AI-driven language learning has also gained attention in recent years. Studies by Li et al. (2020) demonstrate that AI-powered feedback on pronunciation and grammar can significantly improve learning outcomes by providing personalized and immediate

corrective guidance. Integrating AI with real-time communication enhances learners' ability to self-correct and monitor their progress effectively.

From a system development perspective, real-time communication technologies such as **WebRTC** and **Socket.io** have become critical for building scalable platforms that support live interactions. WebRTC enables peer-to-peer audio and video streaming, while Socket.io ensures real-time messaging and notifications. Combining these technologies with a **MERN stack** backend allows developers to create robust, scalable, and user-friendly web applications.[4]

In summary, existing literature underscores the need for an integrated platform that combines real-time communication, AI-powered feedback, and social networking features to enhance language learning. **LetsTalk** addresses these gaps by providing a full-stack, interactive, and scalable solution for learners seeking practical language practice and global collaboration, improving upon platforms like Free4Talk.[5]

CHAPTER 3

SYSTEM ANALYSIS

3.1 Project Analysis

The project analysis is the first step in understanding the requirements and objectives of the **LetsTalk** platform. This analysis includes assessing the system's functional and non-functional requirements, such as user needs, system performance, real-time communication, scalability, and security considerations. It ensures that the platform meets the needs of language learners while maintaining efficiency and reliability.

3.1.1 Requirement Analysis

The primary goal of requirement analysis is to identify and define both the functional and non-functional requirements of the **LetsTalk** system. Functional requirements focus on features that users interact with directly, while non-functional requirements ensure system performance, usability, security, and scalability. This analysis helps ensure that the platform provides an engaging, interactive, and secure environment for language practice.

I. Functional Requirements:

1. **User Registration and Authentication:** Users should be able to register and log in securely using Google or Facebook, with options for profile management and secure logout.
2. **Room Creation and Joining:** Users can create private or public rooms, set language, topic, and level, and join existing rooms for practice.

3. **Real-Time Voice and Video Communication:** Users can participate in live conversations using voice or video, with support for multiple participants per room.
4. **Text Chat:** Users can send messages within rooms or privately, with features like message notifications and read status.
5. **Friend Requests and Following:** Users can connect with other learners, send friend requests, follow users, and view ratings and reviews.
6. **AI-Powered Feedback:** The system provides personalized feedback on pronunciation, grammar, and fluency based on user interactions.
7. **User Profiles:** Users can manage their profile details, track learning progress, view sessions attended, and manage notifications.
8. **Notifications:** The system sends real-time alerts for messages, friend requests, session reminders, and system updates.
9. **Admin Dashboard:** Administrators can manage users, rooms, messages, and reports to ensure smooth operation of the platform.

Use Case Diagram



Figure 3.1 Use Case diagram of User

Table 3.1: Use Case Scenario for user operations

Use Case	User Operations
Primary Actor	User (Person who wants to practice a language or join a conversation)
Secondary Actor	LetsTalk System
Flow of Events	1. User wants to practice a language or join a room. 2. User accesses the LetsTalk platform via web or mobile. 3. User logs in or registers (if not already registered). 4. User browses available rooms or creates a new room. 5. User joins the selected room. 6. User participates in a live voice/video conversation. 7. User can send text messages, friend requests, or follow other users. 8. System provides AI-powered feedback on pronunciation, grammar, and fluency. 9. User receives notifications for messages, friend requests, or session reminders.
Pre-Condition	- User must have internet access and a device capable of audio/video communication. - User must be registered and logged into the LetsTalk system. - User activity (sessions, messages, interactions) is recorded.
Post-Condition	- User receives AI feedback and notifications. - User profile is updated with session statistics, ratings, and reviews. - Real-time response for audio/video streaming.
Quality	- Accurate delivery of messages and notifications. - AI feedback provides clear, actionable guidance.

Use Case Diagram



Figure 3.2 Use Case Diagram of Admin

Table 3.2: Use Case Scenario for Admin operations

Use Case	Admin Operations
Primary Actor	Admin (Person who manages users, rooms, and system operations)
Secondary Actor	LetsTalk System
Flow of Events	1. Admin logs into the LetsTalk system. 2. Admin accesses the dashboard. 3. Admin monitors active rooms and users. 4. Admin manages users (view, edit, block, or delete). 5. Admin manages rooms (approve, edit, delete, or monitor). 6. Admin reviews messages and reports inappropriate content. 7. Admin sends system notifications or announcements. 8. Admin generates reports on user activity, room usage, and system performance.
Pre-Condition	- Admin must have internet access and a device capable of accessing the system. - Admin must be registered and logged into the LetsTalk system. - User activity and rooms are monitored and managed. - Any violations or issues are resolved.
Post-Condition	- Reports and statistics are updated for system performance evaluation.
Quality	- All admin operations must be accurate and responsive. - System should provide valid feedback for each admin action.

II. Non-Functional Requirements:

1. **Performance:** The platform should handle multiple concurrent users without delays in real-time communication.
2. **Scalability:** The system should be able to accommodate growing numbers of users, rooms, and messages.
3. **Security:** User data must be protected with proper authentication, authorization, and encryption mechanisms.
4. **Usability:** The interface should be intuitive, responsive, and accessible across web browsers.
5. **Reliability:** Real-time communication and notifications should be consistent and robust, minimizing downtime.

3.1.2 Feasibility Study

i. Technical Feasibility

The LetsTalk platform, built using the **MERN stack** (MongoDB, Express.js, React, Node.js), is highly technically feasible with modern technology. MongoDB efficiently handles user data, room information, messages, and session statistics, while Express.js and Node.js manage backend logic and server requests. React ensures a dynamic and responsive user interface. Real-time communication is facilitated through **WebRTC** and **Socket.io**, enabling seamless voice, video, and chat interactions.

The system can easily scale using cloud services for hosting and database management. Additional features such as AI-powered feedback, notifications, and friend/follow management can be integrated using third-party APIs and libraries.

Hardware Requirements:

- **Development:** Laptop/desktop with at least Intel i5 or higher, 8GB+ RAM, and stable internet.
- **Production:** Server specifications depend on expected traffic, with options for cloud hosting (AWS, Heroku).

Software Requirements:

- **MongoDB:** For database management of users, messages, and rooms.
- **Express.js:** Backend API framework for handling server requests efficiently.
- **React:** For building responsive user interfaces.
- **Node.js:** For server-side operations and real-time communication handling.
- **WebRTC & Socket.io:** For voice/video and chat real-time communication.
- Cloud hosting and AI integration services can ensure scalability and adaptability.

ii. Operational Feasibility:

The operational feasibility of LetsTalk is favorable because the system is designed to be user-friendly and interactive. Users can browse rooms, create/join sessions, and communicate via voice, video, or text easily. Admins can manage users, rooms, messages, and system notifications efficiently through an intuitive dashboard.

The platform is optimized for both desktop and mobile usage. Minimal training is required for admins to operate the system, while end-users can engage without technical expertise. The real-time communication system, combined with AI feedback, ensures high engagement and smooth operations.

iii. Economic Feasibility:

Economically, LetsTalk provides a cost-effective solution. The **MERN stack and other tools are open-source**, reducing development costs. Cloud-based infrastructure allows pay-as-you-go scalability based on user activity. Automation of tasks such as session tracking, message management, and AI feedback reduces operational costs and human errors.

Enhanced user engagement and interactive features can increase platform adoption, attracting more users and potentially generating revenue through subscriptions, premium features, or ads, ensuring a strong return on investment.

iv. Schedule Feasibility:

The project is feasible within a reasonable timeline, considering the team's expertise with the MERN stack. The development phase is split into several stages, ensuring focused progress:

Table 3.2: Schedule Feasibility

Task	Start Date	End Date	Duration (Days)
Requirement Gathering	13/08/2025	17/08/2025	5
Design & Development	18/08/2025	17/11/2025	90
Testing	18/11/2025	26/11/2025	9
Implementation	27/11/2025	10/02/2026	74
Documentation	11/12/2025	15/04/2026	126

With these milestones, the project can be completed in under 6 months, making it achievable for a skilled team.

This breakdown ensures that LetsTalk is technically, operationally, economically, and schedule-wise feasible, guaranteeing its successful deployment and adoption.

CHAPTER 4

SYSTEM DESIGN

4.1 Design Overview

System design is a crucial phase in the software development lifecycle where the overall architecture and structure of a computer-based system are conceptualized and specified. System design encompasses a thorough understanding of user needs, technological considerations, and performance requirements.

4.1.1 System Architecture

The system follows a client-server architecture using the MERN stack (MongoDB, Express.js, React, Node.js) and Tailwind CSS for styling. The architecture is divided into three main layers:

Frontend (Client-Side): Built using React for a dynamic and responsive user interface.

Backend (Server-Side): Developed with Node.js and Express.js to handle API requests, business logic, and database interactions.

Database: Uses MongoDB for flexible and scalable data storage.

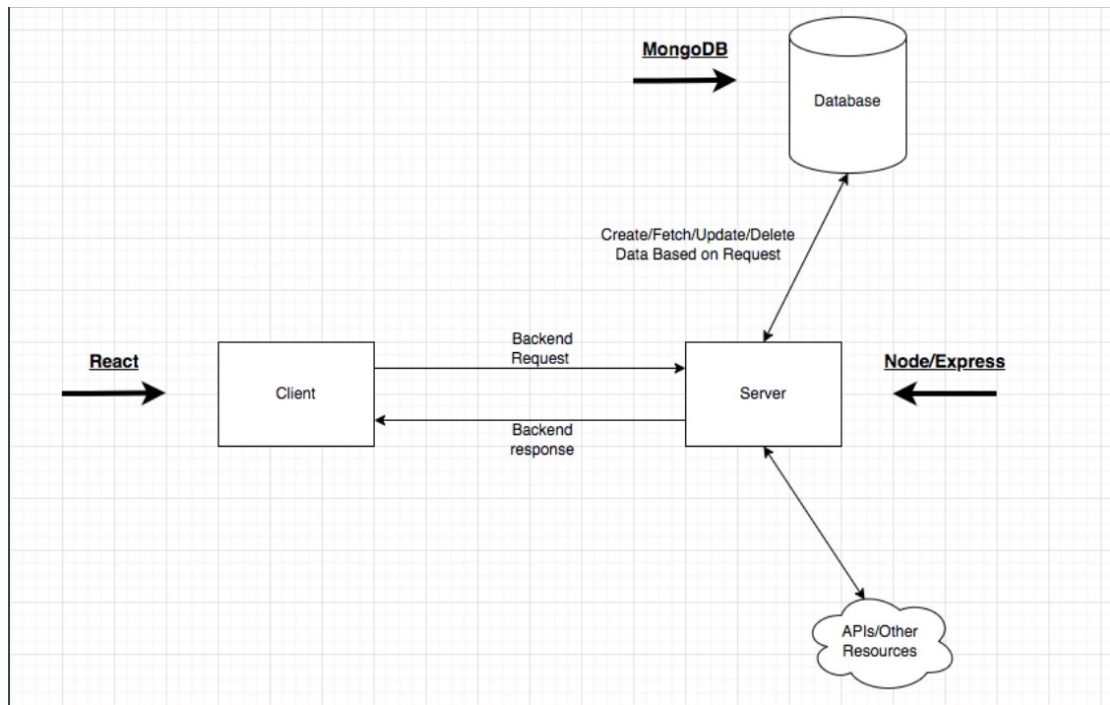


Figure 4.1 System Architecture

4.1.2 Module Design

The LetsTalk system is divided into functional modules, each responsible for specific tasks. These modules interact seamlessly to provide a cohesive and interactive user experience.

User Module:

- Handles user registration, login, and profile management.
- Implements authentication and authorization using **JWT (JSON Web Tokens)**.
- Manages social features like following, friends, and recent activity.

Room Module:

- Manages virtual rooms for language practice, including creation, joining, and leaving rooms.

- Handles room attributes such as language, topic, level, max participants, and tags.
- Monitors room status (live or inactive) and manages participants.

Chat Module:

- Enables real-time text communication between users in rooms or direct messages.
- Integrates **Socket.io** for instant message delivery.
- Supports message history, read receipts, and notifications for unread messages.

Voice & Video Module:

- Implements real-time voice and video communication using **WebRTC**.
- Handles audio/video preferences, including auto-adjust volume and device selection.
- Ensures smooth streaming and minimal latency during live conversations.

AI Feedback Module:

- Provides personalized feedback on user language skills, pronunciation, and fluency.
- Collects session data to generate insights for performance improvement.

Notification Module:

- Sends alerts for friend requests, session reminders, ratings, and reviews.
- Handles system notifications and real-time updates.

7. Admin Module:

- Provides admins with a dashboard to manage users, rooms, and reported content.
- Monitors system activity, generates reports, and resolves violations.
- Ensures system security and smooth operational flow.

LetsTalk Module Interaction Diagram

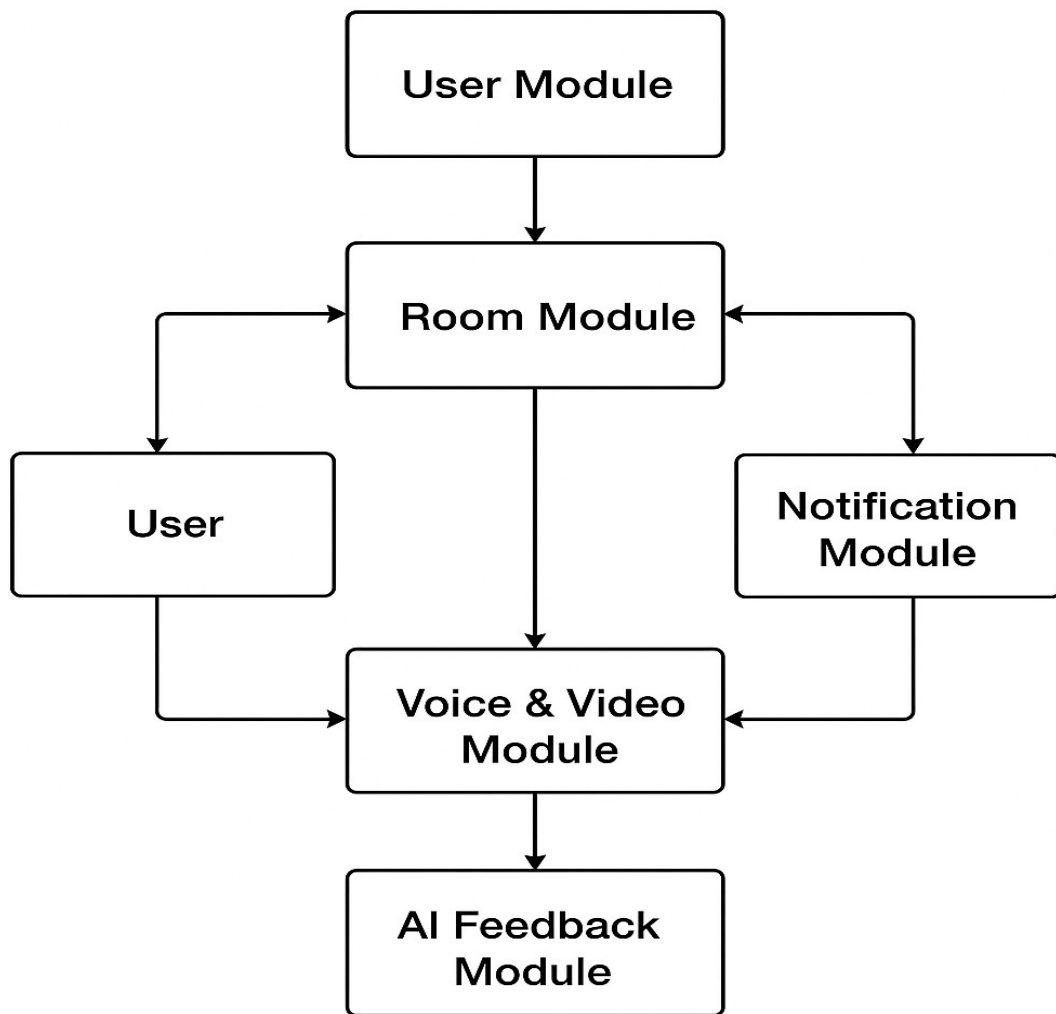


Figure 4.2 Module Interaction Diagram

4.1.3 Database Design

The database for LetsTalk is designed using **MongoDB**, a NoSQL database that provides flexibility, scalability, and efficient handling of real-time data. The database is structured into several key collections to manage users, rooms, messages, and interactions effectively.

Users Collection:

- Stores user information such as name, email, profile photo, provider details (Google/Facebook), and preferences.
- Contains social data like followers, friends, and recent activity.
- Includes learning and native language details and AI feedback history.

Rooms Collection:

- Contains details about each virtual room, including room ID, title, topic, language, level, maximum participants, and tags.
- Tracks live status, active participants, and supports attributes for audio, video, and text communication.

ChatMessages Collection:

- Records all messages sent between users, including sender ID, receiver ID, message content, timestamp, and read status.
- Supports indexing for faster retrieval of chat history and conversation threads.

Notifications Collection:

- Stores notification details for users, including type (message, friend request, session reminder, system alert), title, description, and additional data like room ID or user ID.
- Tracks read/unread status to manage user alerts efficiently.

AI Feedback Collection:

- Records performance metrics and feedback provided to users based on their language practice sessions.
- Includes evaluation details for pronunciation, fluency, and overall session participation.

Admin Collection:

- Contains administrative data for managing users, rooms, and system activity.
- Tracks reported issues, content moderation, and other operational actions.

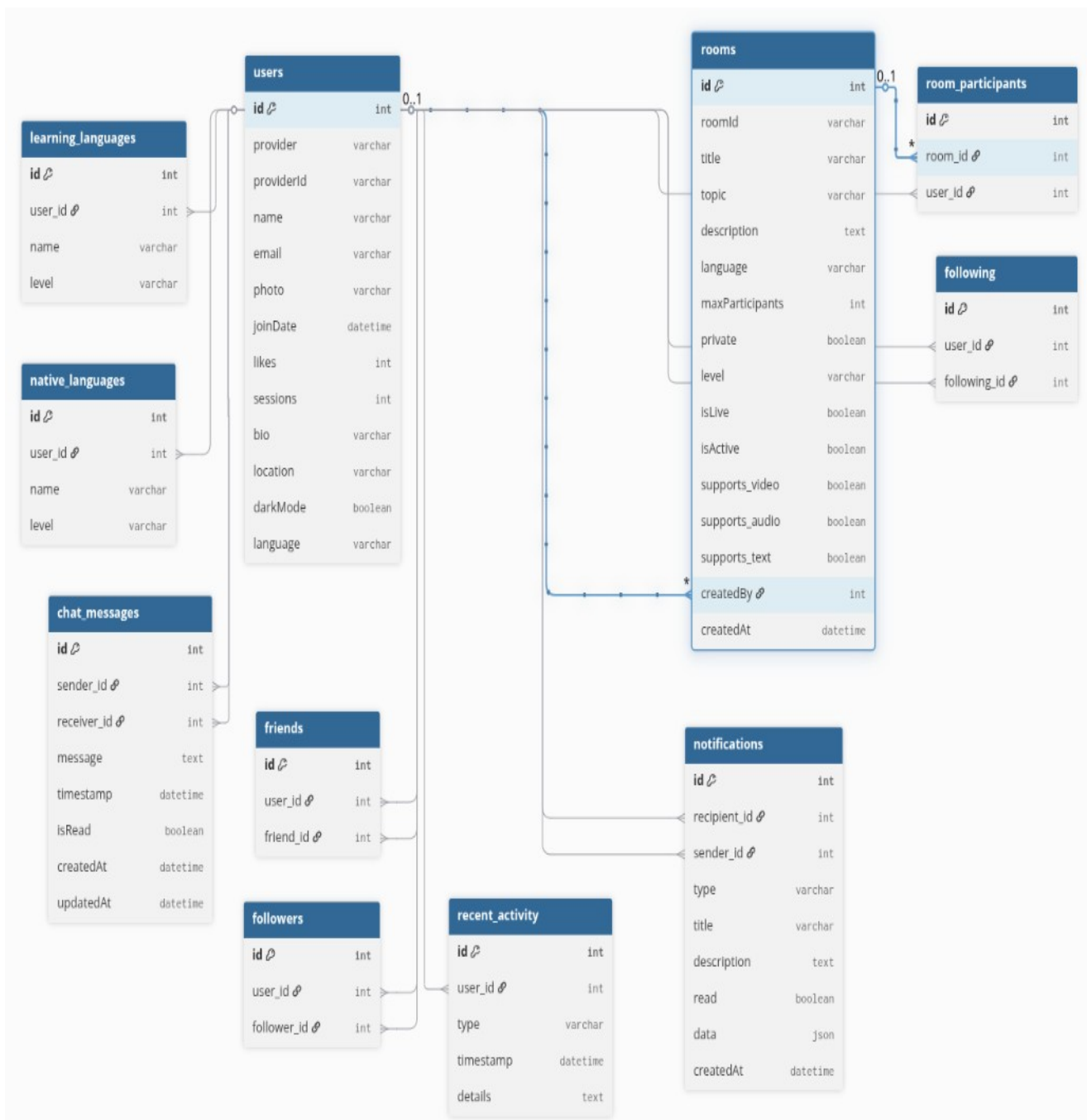


Figure 4.3: Entity-Relationship Diagram

4.1.4 User Interface Design

The user interface is designed to be intuitive and user-friendly, ensuring a seamless experience for LetsTalk users.

Homepage: Displays available language rooms, featured sessions, and suggested rooms based on user interests. Users can browse by language, level, or topic and join active sessions directly from the homepage.

Room Page: Shows details of the selected room, including the title, topic, language, level, maximum participants, and participant list. Users can join voice, video, or text sessions and see real-time participant activity.

Chat & Video/Audio Page: Provides an interface for real-time communication. Users can send text messages, make voice calls, or participate in video conversations within the room. The design ensures smooth interaction and minimal latency.

User Profile Page: Allows users to manage personal information, update preferences, view session history, and track AI feedback on their performance. Users can also manage friends, followers, and social interactions.

Notification Panel: Displays real-time alerts for friend requests, session invitations, ratings, reviews, and system notifications. Users can mark notifications as read or take action directly from the panel.

Admin Dashboard: Enables administrators to monitor active sessions, manage users and rooms, handle reported content, and generate reports on system usage. The dashboard provides a centralized view for efficient system management.

4.1.5 Security Considerations

Security is a critical aspect of the system. The following measures are implemented to ensure data protection and user privacy:

Authentication: Uses JWT (JSON Web Tokens) for secure user authentication. Tokens are stored in HTTP-only cookies to prevent XSS attacks.

Authorization: Implements Role-Based Access Control (RBAC) to restrict access to admin features.

Data Protection: Encrypts sensitive data such as passwords and payment information using bcrypt and SSL/TLS. Ensures secure communication between the client and server using HTTPS.

Input Validation: Validates user inputs to prevent SQL injection and other common attacks.

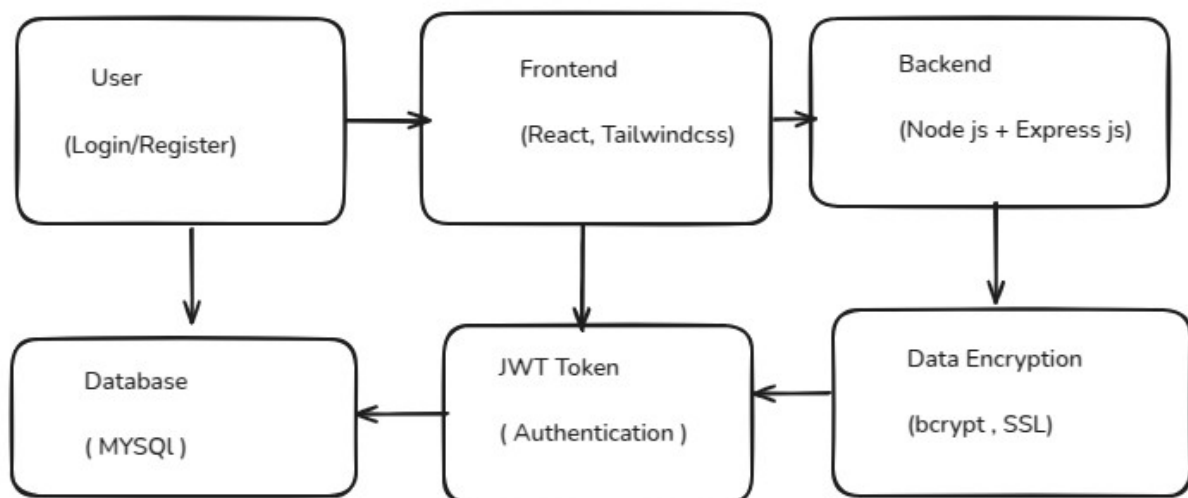


Figure 4.4 : Security Flow Diagram

4.2 Algorithm Details

The LetsTalk platform employs several algorithms to optimize real-time communication, session management, and personalized user experience.

Room Matching Algorithm: Matches users to appropriate language rooms based on their selected language, proficiency level, and interests. The algorithm considers room capacity, live status, and past participation to suggest optimal rooms for engagement.

Real-Time Communication Algorithm: Utilizes WebRTC and Socket.io to establish peer-to-peer connections for audio, video, and text communication. The system dynamically manages network latency and bandwidth to ensure smooth interactions and minimal delay.

Chat Management Algorithm: Handles message delivery, storage, and indexing. Messages are timestamped, marked as read/unread, and efficiently retrieved using sender/receiver indexes for fast conversation loading.

Notification Algorithm: Generates and delivers notifications for friend requests, session reminders, ratings, reviews, and system alerts. Notifications are prioritized and marked as read/unread to maintain a responsive user experience.

User Recommendation Algorithm: Suggests rooms, friends, or sessions using collaborative filtering and activity patterns. It enhances engagement by connecting users with relevant sessions and participants based on past behavior and preferences.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation phase of LetsTalk involves translating the system design into a fully functional web application. This section details the implementation of the platform using the MERN stack (MongoDB, Express.js, React, Node.js), along with WebRTC, Socket.io, and Tailwind CSS. The implementation is divided into frontend, backend, and database components.

5.1.1 Frontend Implementation

The frontend is built using React for a dynamic and responsive user interface. Key components include:

The frontend is built using React to provide a dynamic and responsive user interface. Key components include:

1. Homepage:

- Displays available language rooms, featured sessions, and suggested rooms.
- Users can browse by language, level, or topic and join active sessions.

Example Code:

```
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <BrowserRouter>
          <Routes>
            <Route path="/" element={<Rooms />} />
            <Route path="/rooms" element={<Rooms />} />
            <Route path="/auth" element={<Auth />} />
            <Route path="/buy-me-coffee" element={<Coffee />} />
            <Route path="/privacy-policy" element={<PrivacyPolicy />} />

            <Route element={<ProtectedRoute />>
              <Route path="/settings" element={<Settings />} />
              <Route path="/notifications" element={<Notifications />} />
              <Route path="/profile" element={<Profile />} />
              <Route path="/room/:roomId" element={
                <ErrorBoundaryWrapper>
                  <Room />
                </ErrorBoundaryWrapper>
              } />
              <Route path="/friends" element={<Friends />} />
              <Route path="/chat" element={<Chat />} />
              <Route path="/chat/:userId" element={<Chat />} />
            </Route>
          </Routes>
          <Toaster />
        </BrowserRouter>
      </AuthProvider>
    </QueryClientProvider>
  );
}
```

Figure 5.1: Homepage Example

2. Room Page:

- Shows room details, including title, topic, language, level, max participants, and participants list.
- Users can join voice, video, or text sessions.

Example Code:

```
return (
  <ThemeProvider defaultTheme="light" storageKey="language-app-theme">
    <div className="min-h-screen bg-background text-foreground flex flex-col">
      <div className="container mx-auto p-2 flex-grow">
        <Header onMenuClick={() => setIsSidebarOpen(!isSidebarOpen)} />

        <div className="flex mt-4">
          <Sidebar
            isOpen={isSidebarOpen}
            onClose={() => setIsSidebarOpen(false)}
          />

          <main className="flex-1 lg:ml-0">
            <div className="p-4 lg:p-6">
              {/* Page Header */}
              <div className="flex flex-col gap-4">
                <h1 className="text-3xl font-bold">Language Rooms</h1>
                <p className="text-muted-foreground">
                  Join conversations and practice languages with speakers from
                  around the world. All rooms support video, voice, and text
                  chat.
                </p>
              </div>

              {/* Search and Filters */}
              <div className="flex flex-col sm:flex-row gap-4">
                <div className="relative flex-1">
                  <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 h-4 w-4 text-muted-foreground" />
                  <Input
                    placeholder="Search rooms or topics..."
                    value={searchQuery}
                    onChange={(e) => setSearchQuery(e.target.value)}
                    className="pl-10"
                  />
                </div>

                <Select
                  value={selectedLanguage}
                  onChange={setSelectedLanguage}
                >
                  <SelectTrigger className="w-full sm:w-48">
                    <Filter className="h-4 w-4 mr-2" />
                    <SelectValue placeholder="Filter by language" />
                  </SelectTrigger>
                  <SelectContent>
```

Figure 5.2: Room Page Example

3. Chat & Video/Audio Page:

- Provides real-time text, audio, and video communication.
- Messages are sent, received, and displayed with minimal latency.

Example Code:

```
return (
  <div className="flex h-screen bg-slate-900 text-slate-100 font-sans relative overflow-hidden">
    <Toaster />
    <div className="flex-1 flex flex-col relative">
      <Button
        variant="ghost"
        size="icon"
        className="md:hidden absolute top-4 right-4 z-30 text-slate-100"
        onClick={toggleSidePanel}
        aria-label={isSidePanelOpen ? 'Close side panel' : 'Open side panel'}
      />
      {isSidePanelOpen ? <X className="h-6 w-6" /> : <Menu className="h-6 w-6" />}
    </Button>
    <div className="absolute top-4 left-1/2 -translate-x-1/2 z-20 flex flex-col sm:flex-row gap-2">
      <MediaControls
        isAudioEnabled={isAudioEnabled}
        isVideoEnabled={isVideoEnabled}
        isScreenSharing={isScreenSharing}
        localStream={localStream}
        streamError={streamError}
        onToggleAudio={toggleAudio}
        onToggleVideo={toggleVideo}
        onScreenShare={startScreenShare}
        onLeaveRoom={handleLeaveRoom}
      />
    </div>
    <div className="flex-1 flex items-center justify-center w-full h-full">
      <VideoGrid
        localStream={localStream}
        remoteStreams={remoteStreams}
        users={users}
        isVideoEnabled={isVideoEnabled}
        isAudioEnabled={isAudioEnabled}
        localUserId={stableUserData.id}
      />
    </div>
    {streamError && (
      <div className="absolute top-20 left-1/2 -translate-x-1/2 bg-red-500 text-white p-2 rounded z-30">
        {streamError}
      </div>
    )}
  </div>
)
```

Figure 5.3: Chat & Video/Audio Example

4. User Profile Page:

- Allows users to manage personal information, update preferences, and view session history.
- Displays AI feedback and performance summaries.

Example Code:

```
/* Languages */
<div>
  <h4 className="font-medium mb-2">Speaking Languages</h4>
  <div className="flex flex-wrap gap-1">
    {userData.nativeLanguages.map((lang, index) => (
      <Badge
        key={index}
        className="bg-green-100 text-green-800 dark:bg-green-900 dark:text-green-300"
      >
        {`${lang.name} (${lang.level.charAt(0).toUpperCase() + lang.level.slice(1)})`}
      </Badge>
    ))}
    {userData.nativeLanguages.length === 0 && (
      <Badge className="bg-green-100 text-green-800 dark:bg-green-900 dark:text-green-300">
        English (Native)
      </Badge>
    )}
  </div>
</div>

<div>
  <h4 className="font-medium mb-2">Learning Languages</h4>
  <div className="flex flex-wrap gap-1">
    {userData.learningLanguages.map((lang, index) => (
      <Badge key={index} variant="outline">
        {`${lang.name} (${lang.level.charAt(0).toUpperCase() + lang.level.slice(1)})`}
      </Badge>
    ))}
    {userData.learningLanguages.length === 0 && (
      <Badge variant="outline">French (A2)</Badge>
    )}
  </div>
</div>

<div className="flex items-center gap-2">
  <Star className="h-4 w-4 text-yellow-500" />
  <span className="text-sm">
    {userData.likes} likes • {userData.sessions.toString()} sessions
  </span>
</div>

<div className="flex items-center gap-2">
  <Calendar className="h-4 w-4 text-muted-foreground" />
  <span className="text-sm">
    Total hours: {userData.stats.totalHours.toFixed(2)}
  </span>
</div>
```

Figure 5.4: User Profile Example

5. Notification Panel:

- Displays real-time notifications for friend requests, session invitations, ratings, reviews, and system alerts.

Example Code:

```
return (
  <Dialog open={isOpen} onOpenChange={onClose}>
    <DialogContent className="max-w-2xl">
      <DialogHeader>
        <div className="flex items-center gap-3 mb-2">
          <div className="p-2 rounded-full bg-muted">
            <Icon className="h-5 w-5 text-muted-foreground" />
          </div>
          <div className="flex-1">
            <DialogTitle className="text-left">{notification.title}</DialogTitle>
            <div className="flex items-center gap-2 mt-1">
              <span className="text-sm text-muted-foreground">{notification.time}</span>
              <Badge
                variant="secondary"
                className={getTypeColor(notification.type)}
              >
                {notification.type}
              </Badge>
              {!notification.read && (
                <Badge variant="default" className="text-xs">
                  New
                </Badge>
              )}
            </div>
          </div>
        </div>
      </DialogHeader>

      <Separator />

      <DialogDescription className="text-left text-base leading-relaxed py-4">
        {notification.description}
      </DialogDescription>

      <Separator />

      <div className="flex items-center justify-between pt-4">
        <div className="flex gap-2">
          {getActionButtons()}
        </div>
      </div>
    </DialogContent>
  </Dialog>
)
```

Figure 5.5: Notification Panel Example

5.1.2 Backend Implementation

The backend is implemented using Node.js and Express.js to handle API requests, business logic, and database operations. Key features include:

1. User Authentication:

- Implements secure authentication using JWT.

Example Code:

```
const jwt = require('jsonwebtoken');

const generateToken = (userId) => {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn: '7d' });
};

module.exports = generateToken;
```

Figure 5.6: Authentication Example

2. Room Management:

- Handles CRUD operations for rooms and session management.

Example Code:

```
const express = require('express');
const router = express.Router();
const { createRoom, getPublicRooms, getRoomById, deleteRoom, getLiveRooms, getFeaturedRooms } = require('../controllers/rooms');
const { isAuthenticated } = require('../middleware/auth.middleware');

router.post('/rooms', isAuthenticated, createRoom);
router.get('/rooms/count', countAllRooms);
router.get('/rooms/featured', getFeaturedRooms);
router.get('/rooms/live', getLiveRooms);
router.get('/rooms', getPublicRooms);
router.get('/rooms/:roomId', isAuthenticated, getRoomById);
router.delete('/rooms/:roomId', isAuthenticated, deleteRoom);

module.exports = router;
```

Figure 5.7: Room Management Example

3. Chat & Message Management:

- Manages real-time messaging using Socket.io.
- Stores chat history in MongoDB with indexing for fast retrieval.

Example Code:

```
module.exports = (io) => {  
  
  io.on('connection', (socket) => {  
    console.log('User connected to chat:', socket.id);  
  
    // User joins chat with their ID  
    socket.on('joinChat', (userId) => {  
      console.log('User joined chat:', userId);  
      userSockets.set(userId, socket.id);  
      socket.userId = userId;  
      socket.join(`user_${userId}`);  
    });  
  
    // Handle private messages  
    socket.on('sendPrivateMessage', async (data) => {  
      try {  
        const { receiverId, message, senderId } = data;  
  
        // Save message to database  
        const chatMessage = new ChatMessage({  
          sender: senderId,  
          receiver: receiverId,  
          message: message.trim()  
        });  
  
        await chatMessage.save();  
        await chatMessage.populate('sender', 'name photo');  
        await chatMessage.populate('receiver', 'name photo');  
  
        // Send to receiver if they're online  
        const receiverSocketId = userSockets.get(receiverId);  
        if (receiverSocketId) {  
          io.to(`user_${receiverId}`).emit('receivePrivateMessage', {  
            message: chatMessage,  
            from: senderId  
          });  
        }  
      }  
    });  
  });  
}
```

Figure 5.8: Chat Management Example

4. AI Feedback & Recommendation System:

- Processes session data to provide user-specific feedback and room suggestions.

Example Code:

```
const users = roomState.users[roomId] || [];
const ownerId = roomState.owners[roomId] || '';
io.to(roomId).emit('roomUsers', { users, ownerId });
console.log(`Emitted roomUsers for room ${roomId}: ${users.length} users, owner: ${ownerId}`);
});

io.on('connection', (socket) => {
  console.log(`\ Client connected:`, socket.id);

  socket.on('joinRoom', async ({ roomId, userId, userName, roomTitle }) => {
    try {
      const roomData = await checkRoomExists(roomId);
      if (!roomData.exists) {
        console.log(`useRoom: Room ${roomId} not found, attempting to create`);
        await Room.create({ roomId, createdBy: new mongoose.Types.ObjectId(userId), participants: [new mongoose.Types.ObjectId(userId)], islive: true, roomData.exists = true;
        roomData.maxParticipants = 10;
      }

      if (socket.roomId === roomId && socket.userId === userId && !roomState.reconnecting.has(socket.id)) {
        console.log(`\ User ${userId} already in room ${roomId} on this socket, emitting current state`);
        emitRoomUpdate(roomId);
        return;
      }

      if (socket.roomId && socket.roomId !== roomId && socket.userId) {
        console.log(`\ Socket switching from room ${socket.roomId} to ${roomId}`);
        const removedUser = await removeUserFromRoom(socket.roomId, socket.userId);
        if (removedUser) {
          socket.leave(socket.roomId);
          const newOwnerId = handleOwnershipTransfer(socket.roomId, socket.userId);
          if (newOwnerId) io.to(socket.roomId).emit('ownershipTransferred', { newOwnerId });
          emitRoomUpdate(socket.roomId);
          io.to(socket.roomId).emit('userLeft', { userId: socket.userId, socketId: socket.id });
        }
      }

      const existingUser = roomState.users[roomId]?.find(u => u.userId === userId);
      if (existingUser && existingUser.socketId !== socket.id) {
        console.log(`\ User ${userId} switching socket in room ${roomId}, updating socketId from ${existingUser.socketId} to ${socket.id}`);
        roomState.reconnecting.add(socket.id);
        const oldSocket = io.sockets.sockets.get(existingUser.socketId);
        if (oldSocket) {
          oldSocket.disconnect(true);
          console.log(`\ Disconnected stale socket ${existingUser.socketId} for user ${userId}`);
        }
      }
    }
  });
});
```

Figure 5.9: AI Feedback Example

5. Notification System:

- Generates and delivers notifications for all user interactions.

Example Code:

```
const express = require('express');
const {
  getUserNotifications,
  markNotificationAsRead,
  markAllNotificationsAsRead,
  deleteNotification,
  createNotificationAPI
} = require('../controller/notification.controller');
const { isAuthenticated } = require('../middleware/auth.middleware');

const router = express.Router();

router.get('/notifications', isAuthenticated, getUserNotifications);
router.post('/notifications', isAuthenticated, createNotificationAPI);
router.patch('/notifications/:notificationId/read', isAuthenticated, markNotificationAsRead);
router.patch('/notifications/mark-all-read', isAuthenticated, markAllNotificationsAsRead);
router.delete('/notifications/:notificationId', isAuthenticated, deleteNotification);

module.exports = router;
```

Figure 5.10: Notification System Example

5.1.3 Database Implementation

The database is implemented using MongoDB, providing flexibility and scalability.

Key collections include:

1. Users Collection:

- Stores user details such as name, email, profile photo, social connections, languages, and preferences.

Schema:

```
const userSchema = new mongoose.Schema({
  provider: {
    type: String,
    enum: ['google', 'facebook'],
    required: true,
  },
  providerId: {
    type: String,
    required: true,
    unique: true,
  },
  name: String,           // Display name
  email: String,
  photo: String,          // Profile photo URL

  joinDate: {
    type: Date,
    default: Date.now,
  },

  // Stats
  likes: { type: Number, default: 0 },
  sessions: { type: Number, default: 0 },
  stats: {
    weeklySessions: { type: Number, default: 0 },
    totalHours: { type: Number, default: 0 },
    languagesPracticed: { type: Number, default: 0 },
  },

  // Social connections
  followers: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  following: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  friends: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],

  // Notifications, Privacy, Audio/Video, Appearance
  notificationPrefs: { "Prefs": Unknown word.
    emailNotifications: { type: Boolean, default: true },
    appNotifications: { type: Boolean, default: true },
  },
  privacyPrefs: { "Prefs": Unknown word.
    showProfilePicture: { type: Boolean, default: true },
    showActivityStatus: { type: Boolean, default: true },
  },
  audioVideoPrefs: { "Prefs": Unknown word.
    preferredMic: String,
    preferredSpeaker: String,
    autoAdjustVolume: { type: Boolean, default: true },
  },
});
```

Figure 5.11: User Schema Code

2. Rooms Collection:

- Contains room details like room ID, title, topic, language, level, participants, and session status.

Schema:

```
const mongoose = require('mongoose');

const roomSchema = new mongoose.Schema({
  roomId: { type: String, required: true, unique: true }, // e.g., CR947

  title: { type: String, required: true },
  topic: { type: String, default: 'General' },
  description: { type: String },

  language: { type: String, required: true },
  maxParticipants: { type: Number, default: 10 },

  private: { type: Boolean, default: false },

  tags: [{ type: String }],

  level: { type: String,
    enum: ['beginner', 'intermediate', 'advanced'],
    default: 'beginner' }, // beginner, intermediate, advanced

  isLive: { type: Boolean, default: true },
  isActive: { type: Boolean, default: true },

  supports: {
    video: { type: Boolean, default: true },
    audio: { type: Boolean, default: true },
    text: { type: Boolean, default: true }
  },

  participants: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],

  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now }
}, { versionKey: false });

// Indexes (optional for filtering)
roomSchema.index({ language: 1 });
roomSchema.index({ tags: 1 });

module.exports = mongoose.model('Room', roomSchema);
```

Figure 5.12: Room Schema Code

3. ChatMessages Collection:

- Stores messages with sender, receiver, timestamp, and read/unread status.

Schema:

```
const mongoose = require('mongoose');

const chatMessageSchema = new mongoose.Schema({
  sender: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  receiver: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  message: {
    type: String,
    required: true,
    trim: true
  },
  timestamp: {
    type: Date,
    default: Date.now
  },
  isRead: {
    type: Boolean,
    default: false
  }
}, {
  timestamps: true
});

// Index for faster queries
chatMessageSchema.index({ sender: 1, receiver: 1, timestamp: -1 });

module.exports = mongoose.model('ChatMessage', chatMessageSchema);
```

Figure 5.13: Chat Message Schema Code

4. Notifications Collection:

- Tracks notifications for friend requests, session updates, ratings, reviews, and system messages.

Schema:

```
const mongoose = require('mongoose');

const notificationSchema = new mongoose.Schema({
  recipient: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  sender: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: false // System notifications won't have a sender
  },
  type: {
    type: String,
    enum: ['message', 'friend', 'session', 'system', 'like', 'follow'],
    required: true
  },
  title: {
    type: String,
    required: true
  },
  description: {
    type: String,
    required: true
  },
  read: {
    type: Boolean,
    default: false
  },
  data: {
    type: mongoose.Schema.Types.Mixed, // Additional data like room ID, user ID, etc.
    default: {}
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Index for efficient queries
notificationSchema.index({ recipient: 1, createdAt: -1 });
notificationSchema.index({ recipient: 1, read: 1 });

module.exports = mongoose.model('Notification', notificationSchema);
```

Figure 5.14: Notification Schema Code

5.2 Testing

5.2.1 Unit Testing

Unit testing verifies that individual components function correctly in isolation. Backend testing is performed using Jest or Mocha/Chai, and frontend testing uses React Testing Library.

Test Case ID	Description	Test Data	Expected Result	Status
TC001	Verify if a user can log in with valid credentials	Email: test@example.com, Password: Test@123	User is successfully logged in and redirected to the dashboard	Pass/ Fail

Table 5.1: Unit Test Case for User Authentication

5.2.2 Integration Testing

Integration testing ensures that multiple modules work together, including API calls, database operations, and UI interactions.

Test Case ID	Description	Steps	Expected Result	Status
TC002	Verify if the cart and checkout process work correctly	1. Add items to the cart 2. Proceed to checkout 3. Enter shipping details 4. Select payment method 5. Place order	Order is successfully placed, and confirmation email is sent	Pass/ Fail

Table 5.2: Integration Test Case for Cart & Checkout

5.2.3 System Testing

System testing validates that the application meets all functional and non-functional requirements.

Test Case ID	Description	Test Data	Expected Result	Status
TC003	Verify payment transactions using eSewa and Khalti	Amount: \$50, Payment Method: Khalti	Payment is successfully processed, and order is confirmed	Pass/ Fail

Table 5.3: System Test Case for Payment Gateway Integration

5.2.4 User Acceptance Testing (UAT)

UAT ensures that the system meets business requirements and user expectations.

Test Case ID	Description	Steps	Expected Result	Status
TC004	Verify if users can track their order status from the dashboard	1. Log in 2. Navigate to "My Orders" 3. Check order history	Order history is displayed correctly with tracking information	Pass/ Fail

Table 5.4: UAT Test Case for Dashboard

5.2.5 Security Testing

Security testing checks for vulnerabilities like **SQL Injection**, **XSS**, and **Brute Force** attacks.

Security Test	Description	Expected Outcome
SQL Injection	Attempt injecting OR '1'='1' in login fields	Input is sanitized, and no unauthorized access is granted
XSS Attack	Inject <code><script>alert('Hacked')</script></code> in input fields	Script is escaped, and no alert is shown
Brute Force Attack	Try multiple incorrect passwords in quick succession	Account locks after 5 failed attempts

Table 5.5: Security Test Cases

5.2.6 Performance Testing

Performance testing evaluates system behavior under various loads.

Test Type	Scenario	Expected Result
Load Testing	1000 users browse and purchase items simultaneously	The system remains responsive
Stress Testing	5000 concurrent users place orders	System gracefully handles load or shows an appropriate message
Response Time	Measure API response time for product search	Should be < 500ms

Table 5.6: Performance Test Cases

5.3 Conclusion

The **LetsTalk** has undergone thorough testing, including **unit, integration, system, UAT, security, and performance** tests. These ensure that the application is **functional, secure, and optimized** for real-world usage. Upon successful UAT, the system will be ready for deployment.

CHAPTER 6

CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

The **LetsTalk** real-time language practice platform has been successfully developed to address the challenges faced by individuals seeking to improve their communication skills through interactive and immersive conversations. The system provides a seamless, user-friendly, and scalable solution for connecting people worldwide to practice languages in real-time.

Key features of the system include:

- **Real-Time Communication:** Users can instantly connect and practice speaking with others through text and voice chat.
- **Multiple Language Support:** Enables users to choose from various languages for practice sessions.
- **User-Friendly Interface:** Simple and intuitive design for quick access to conversations.
- **Profile & Matching System:** Connects users based on language preferences, skill level, and interests.
- **Community Interaction:** Offers open rooms, private chats, and group conversations to suit different learning styles.
- **Secure Communication:** Ensures privacy and safety with authentication and content moderation features.

Throughout the development process, modern web technologies (Node.js, Socket.io, Express.js, and JWT authentication) were utilized to ensure the platform is fast, secure, and scalable.

By implementing **LetsTalk**, language learners can improve fluency, overcome hesitation, and gain cultural exposure through real-life conversations. The platform's flexible architecture ensures that future enhancements such as AI-powered pronunciation feedback, video chat, and gamified learning can be integrated to further enrich the learning experience.

6.2 Future Recommendation

While the **LetsTalk** platform already provides an engaging and efficient environment for real-time language practice, there are several opportunities for future improvements to enhance user experience, accessibility, and learning outcomes.

1. Mobile Application Development

- Develop dedicated Android and iOS apps to make language practice more accessible on the go.
- Implement push notifications for session invites, friend requests, and practice reminders.

2. AI-Powered Language Assistance

- Integrate AI-driven pronunciation analysis to give users instant feedback on speaking skills.

- Use AI to suggest conversation topics and vocabulary tailored to the user's language level.

3. Screen Sharing Integration

- Support screen sharing for language learning materials.

4. Gamification and Rewards

- Introduce points, badges, and leaderboards to encourage consistent participation.
- Offer streak rewards for daily practice.

5. Multi-Language Interface

- Provide the platform's interface in multiple languages to attract a wider global audience.

6. Enhanced Security & Moderation

- Implement multi-factor authentication (MFA) for account safety.
- Use AI-powered moderation tools to detect and prevent abusive content in chats.

7. Advanced Matching Algorithms

- Match users based on shared interests, time zones, and learning goals for more engaging conversations.
- Offer "language exchange" mode where two users teach each other their native languages.

8.

9. Community Features

- Add discussion forums, language clubs, and event hosting for community-driven learning.
- Allow group practice sessions for debate and role-playing activities.

10. Integration with Language Learning Tools

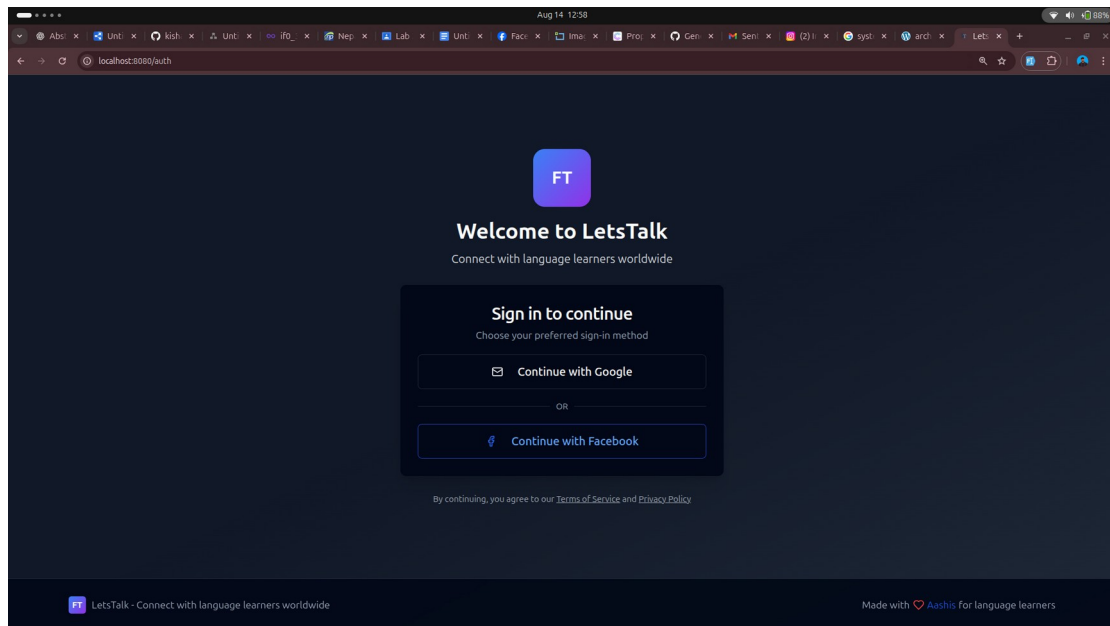
- Partner with dictionary, translation, and grammar correction APIs for real-time assistance during conversations.

By implementing these enhancements, **LetsTalk** can evolve into a world-class language learning community that offers a richer, more personalized, and more secure environment for learners worldwide. These improvements will increase engagement, strengthen the platform's global presence, and ensure long-term scalability.

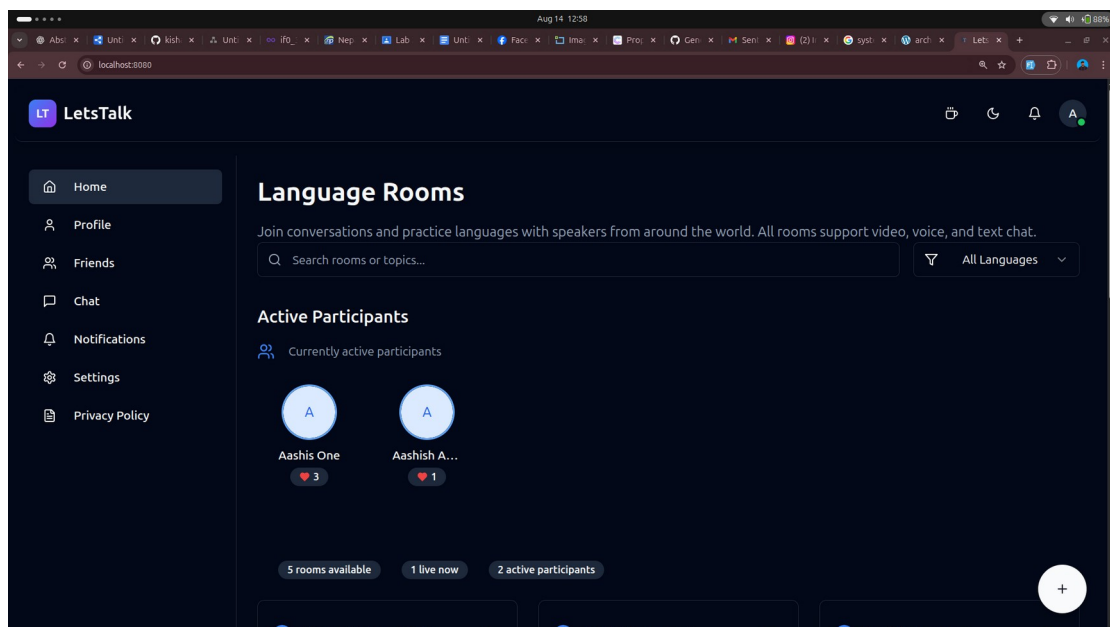
REFERENCES

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [2] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [3] J. Resig and B. Bibeault, *Secrets of the JavaScript Ninja*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2023.
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [5] OWASP Foundation, *OWASP Top Ten: The Ten Most Critical Web Application Security Risks*, 2023. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: 20-Mar-2025].
- [6] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.
- [7] MDN Web Docs, *Web Technologies Documentation: HTML, CSS, and JavaScript*, 2023. [Online]. Available: <https://developer.mozilla.org>. [Accessed: 20-Mar-2025].
- [8] B. Kurniawan, *MongoDB for Jobseekers: NoSQL Database for Beginners*. New Delhi, India: BPB Publications, 2021.

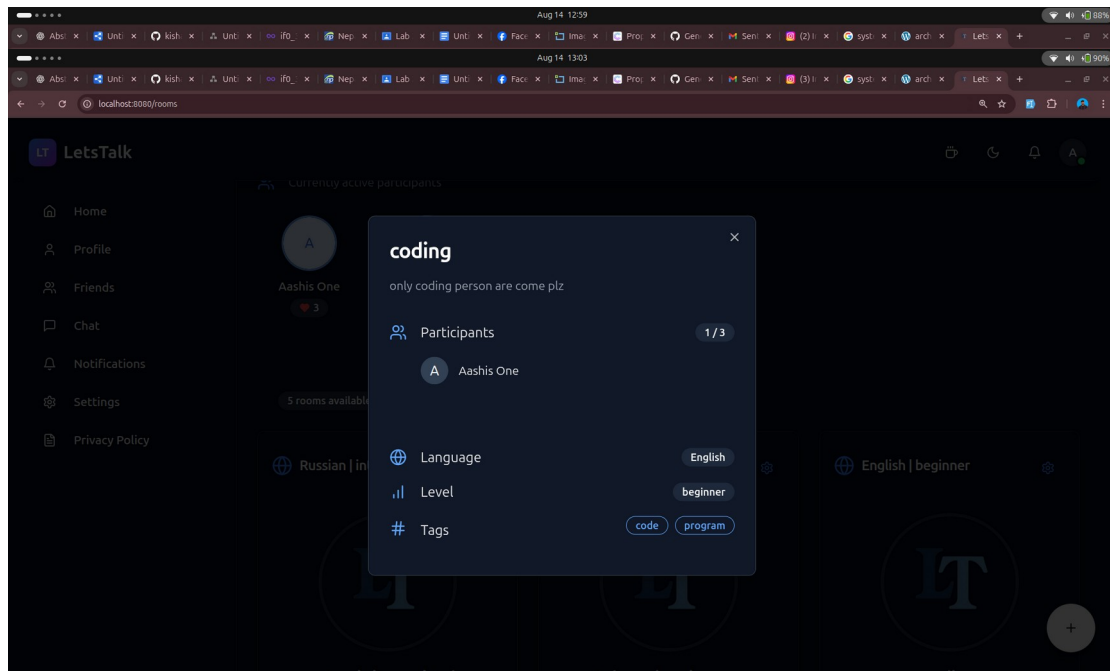
APPENDICES



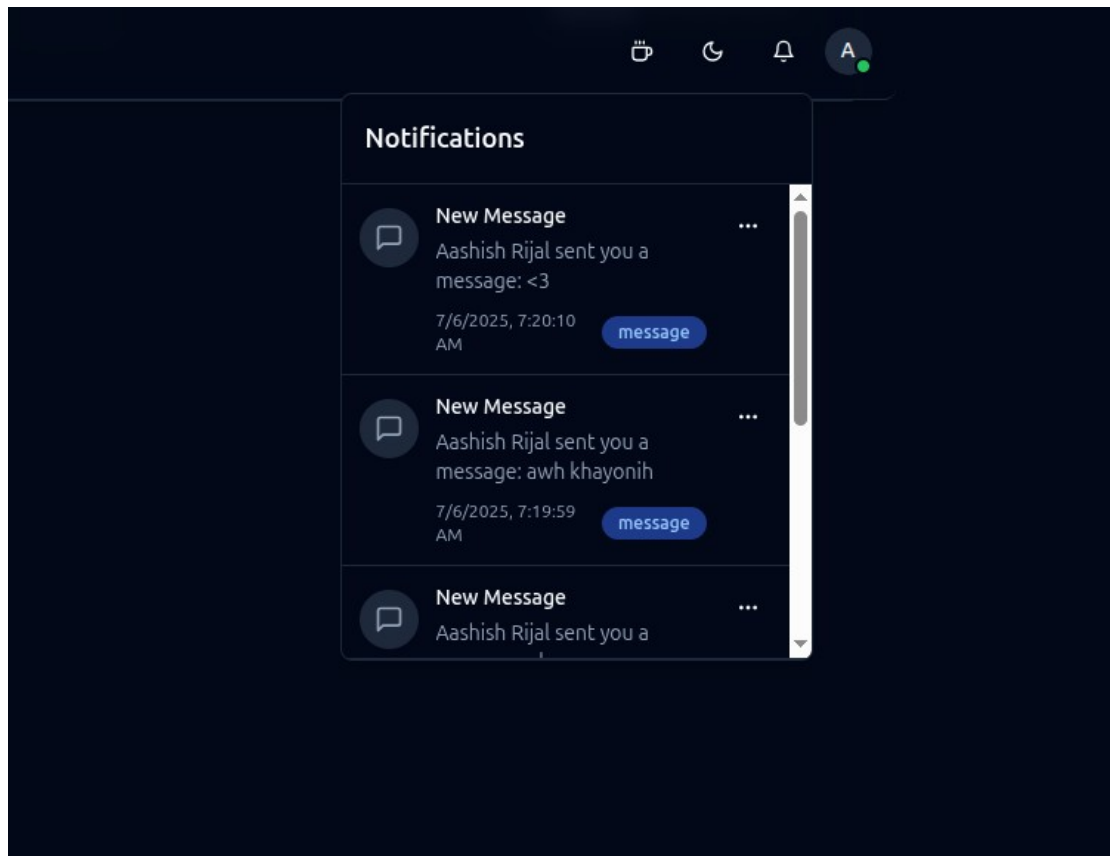
Login/Register page



Homepage



Room Info



Notification Popup

Create New Room

Set up a new language practice room. Participants will have access to video, voice, and text chat.

Room Title

e.g., English Conversation for Beginners

Topic

e.g., Travel, Hobbies, Technology

Description

Describe what participants can expect...

Primary Language

Select a language

Difficulty Level

Beginner

Maximum Participants

10 participants

Private Room

Only invited users can join

Tags (Optional)

Add a tag...

Add

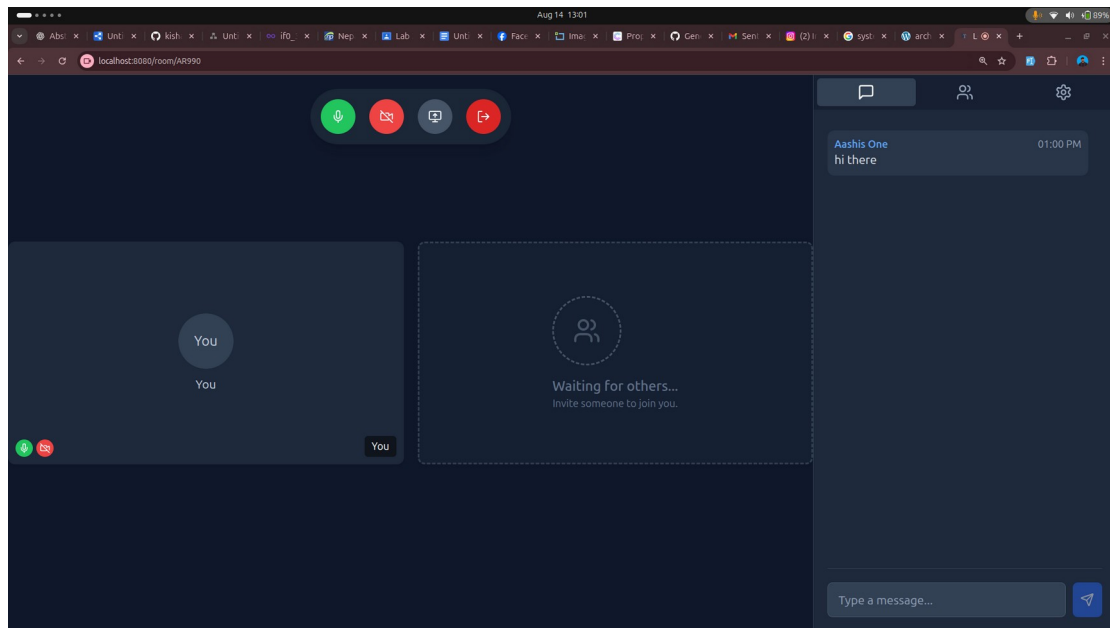
Available Features

All rooms include video calls, voice chat, and text messaging. Participants can toggle between modes during the session.

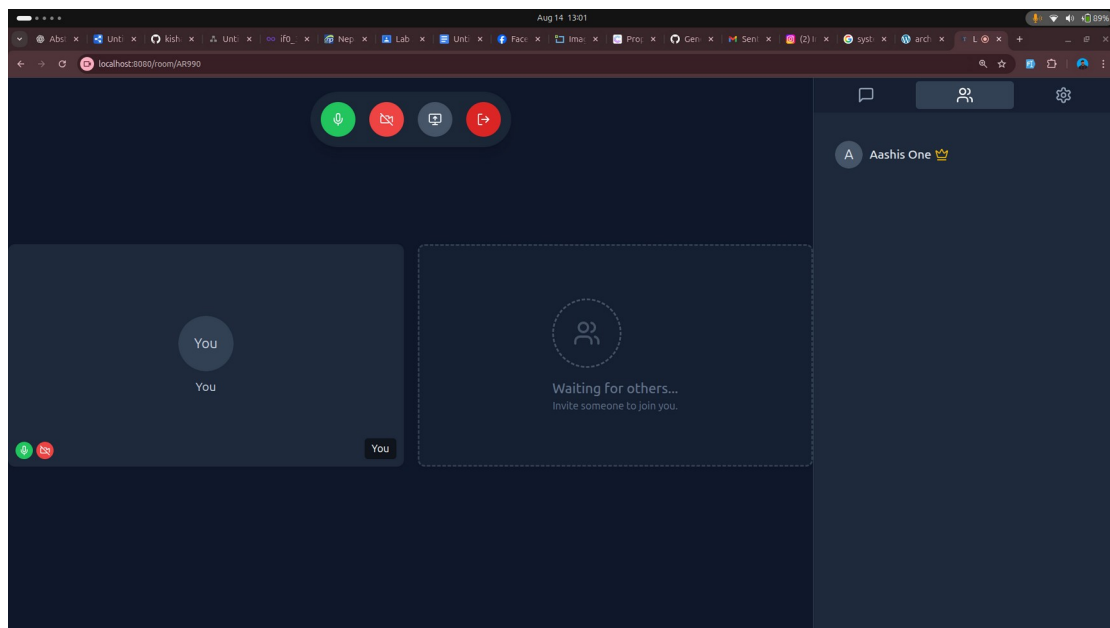
Cancel

Create Room

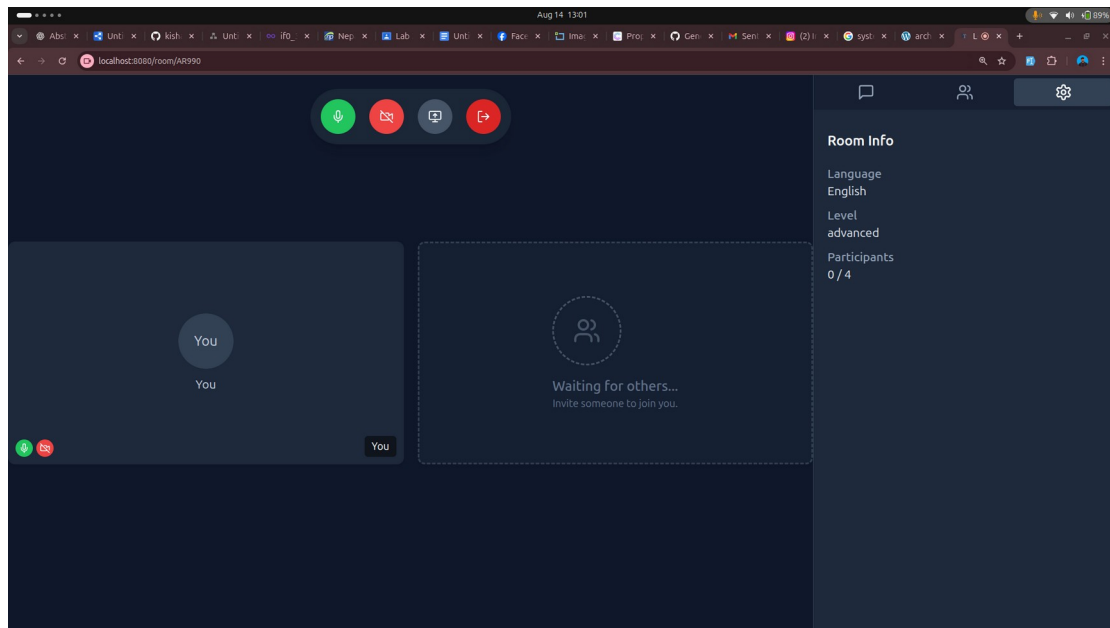
create Room



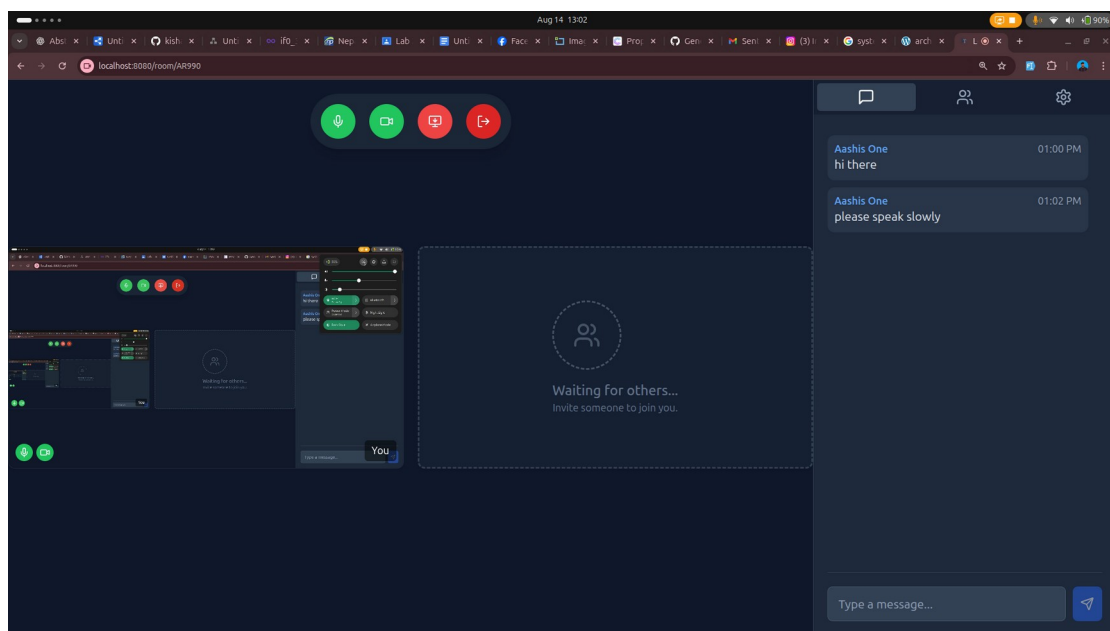
chat/Room



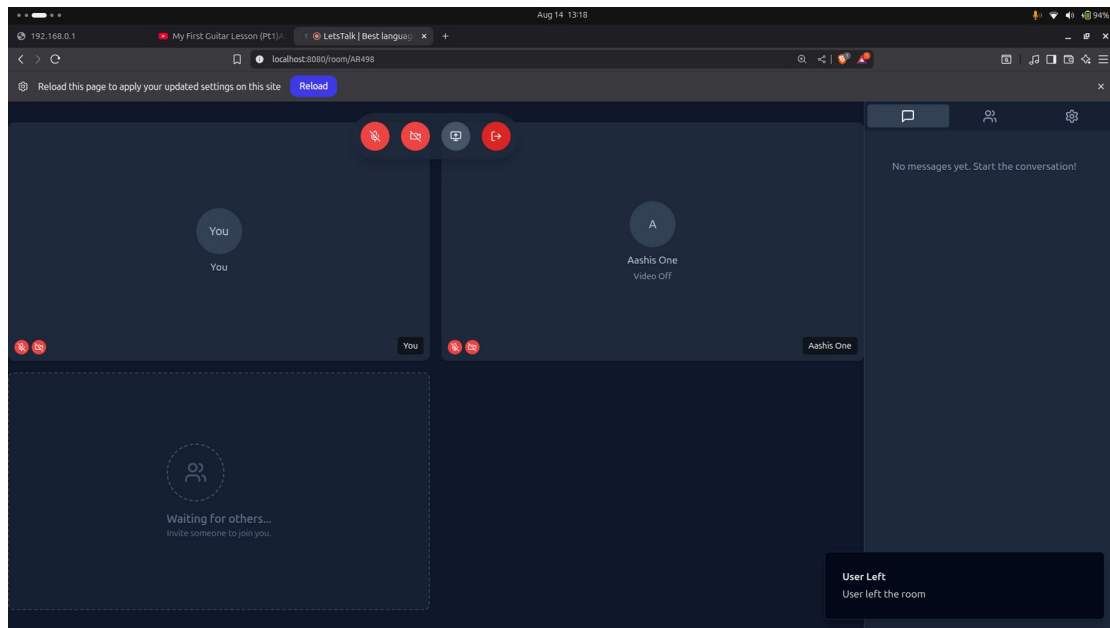
Participants/Room



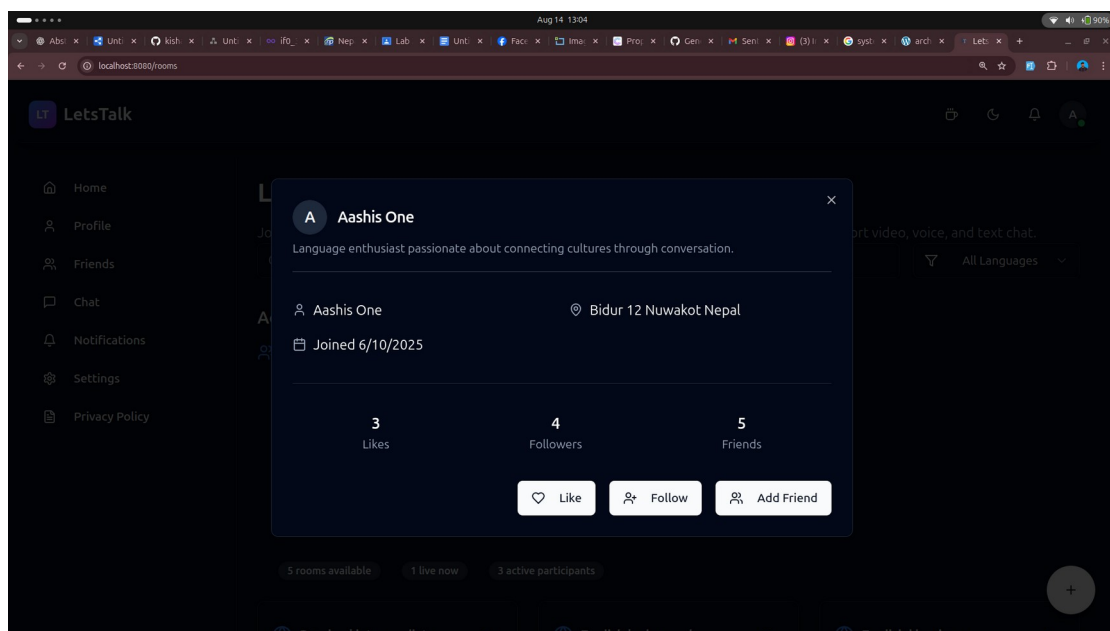
Info/Room



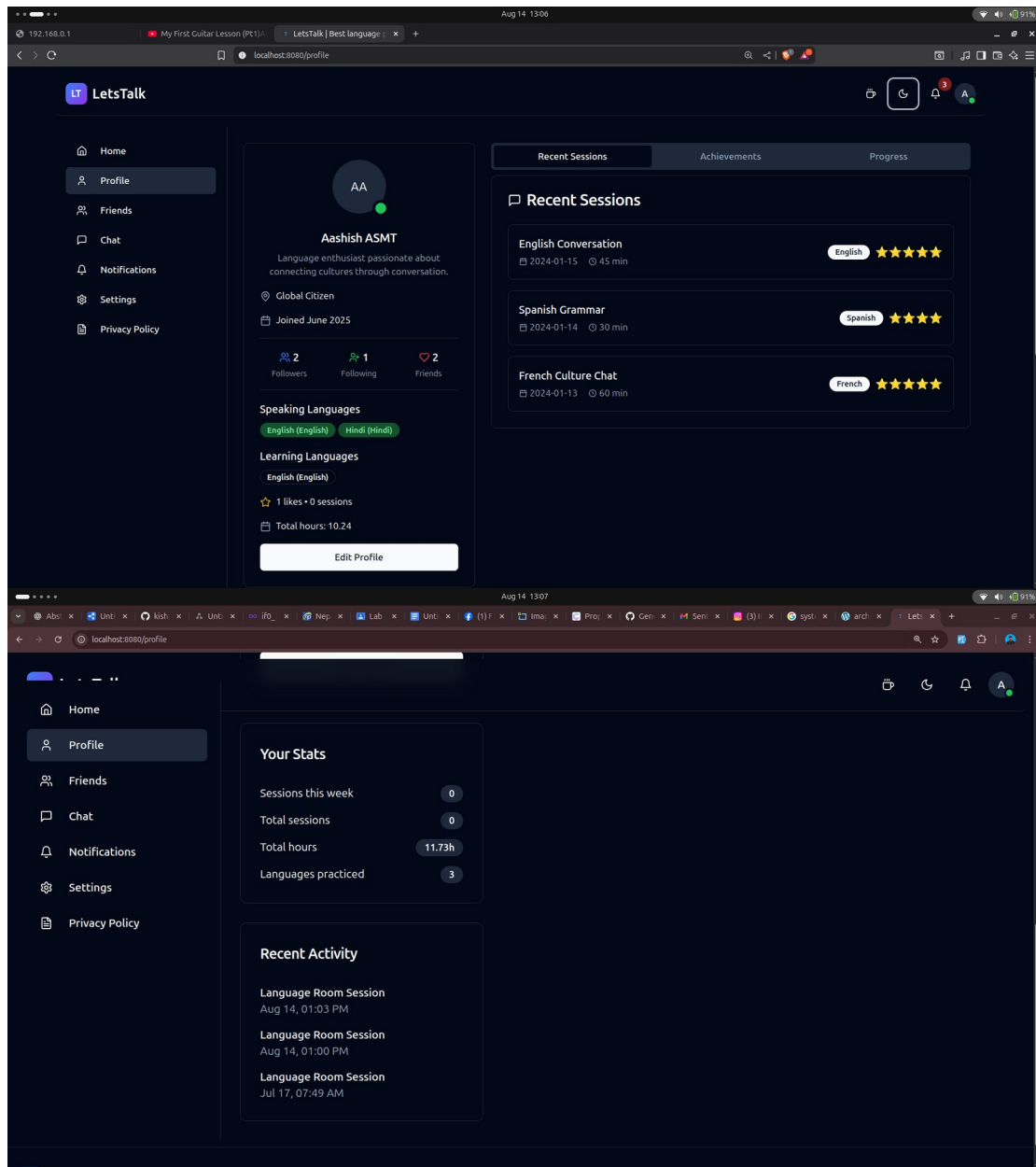
ScreenShare/Room



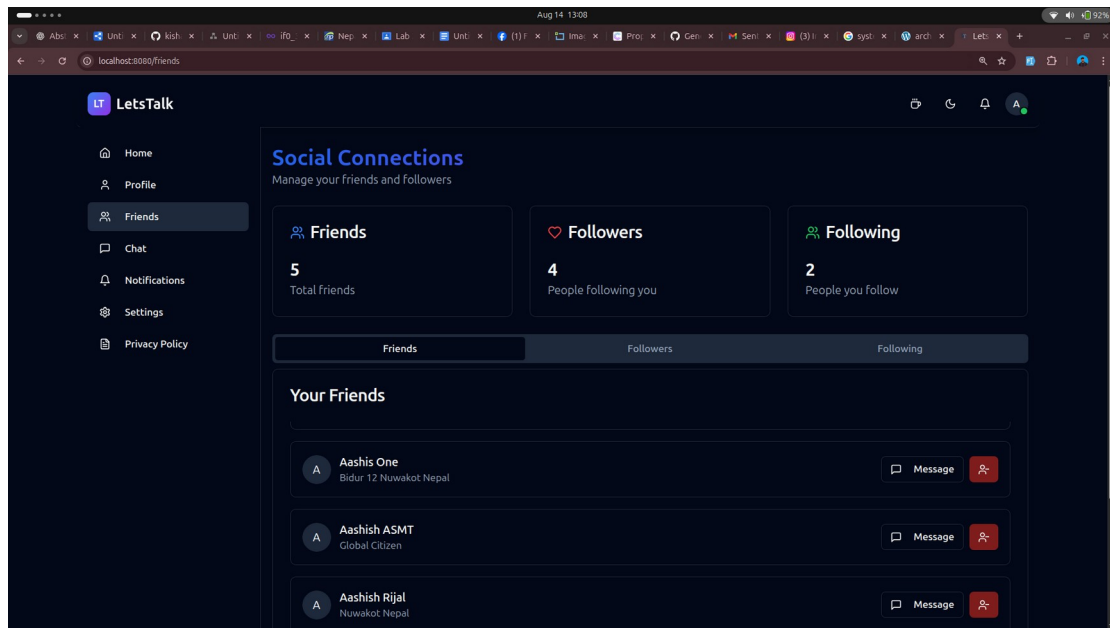
Multiple User Room



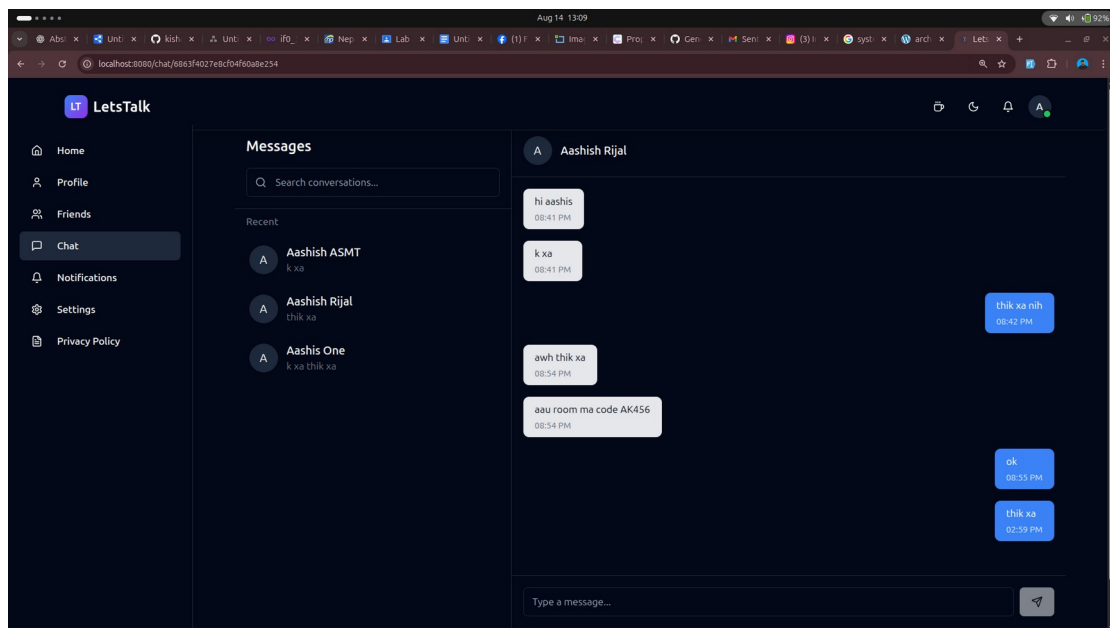
user Info



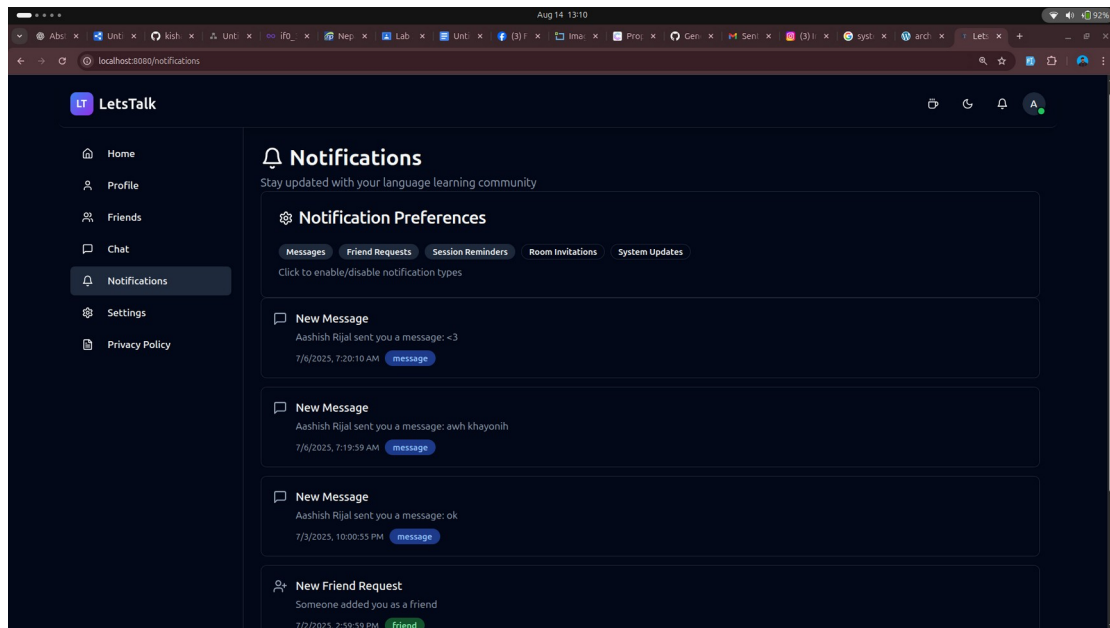
User Profile



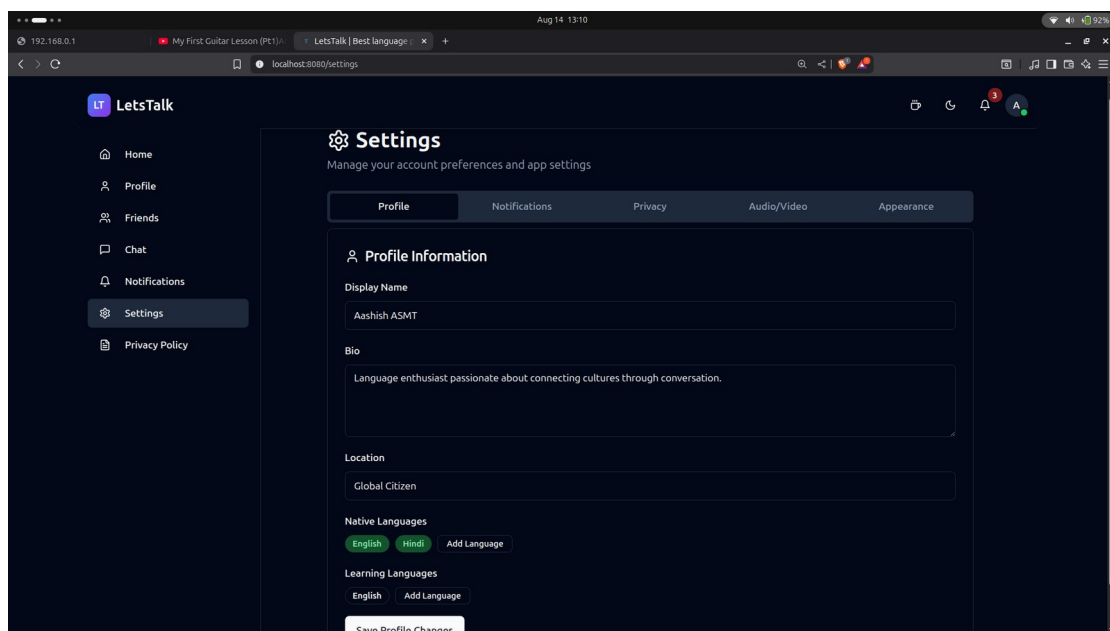
friends



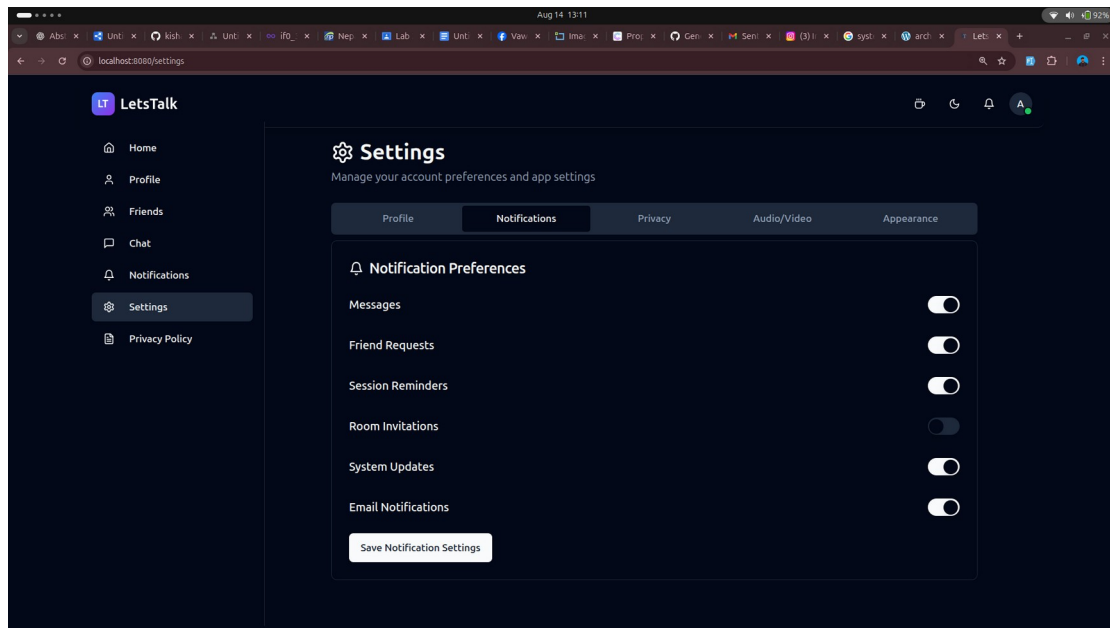
chats



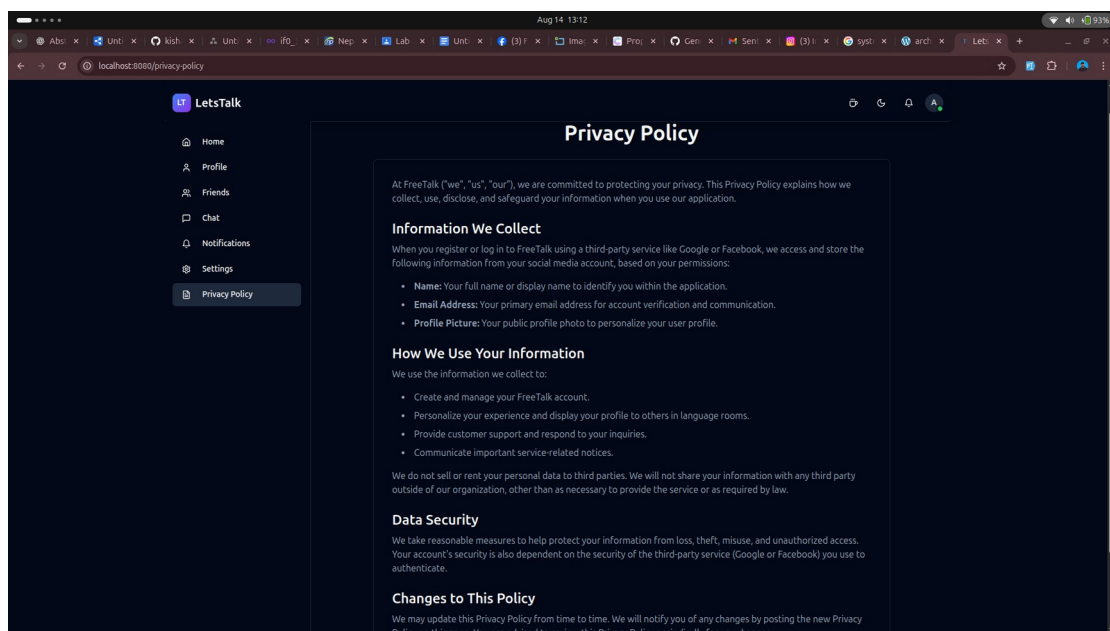
notifications



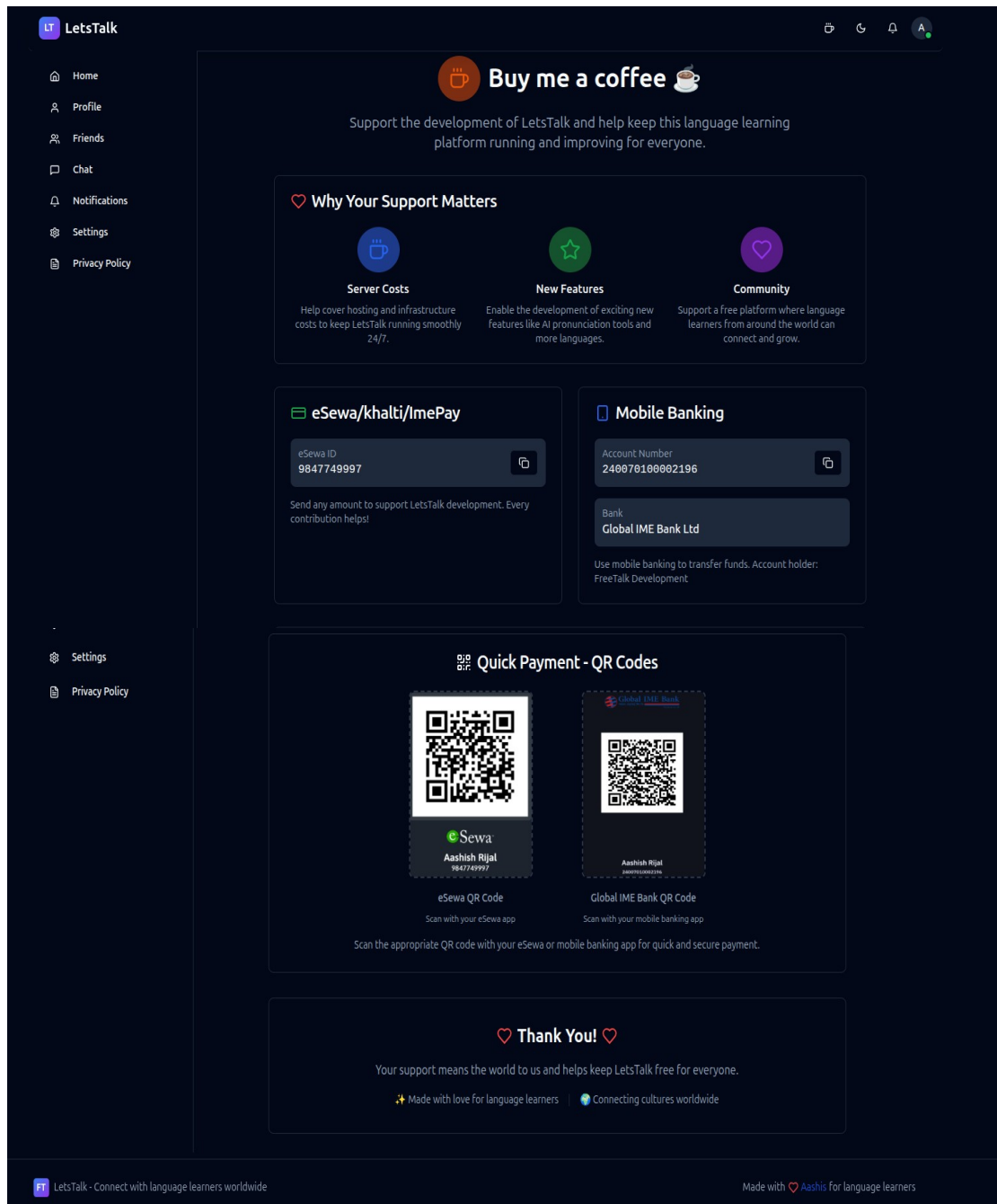
Setting/userProfile



setting/notifications



privacy policy



Buy Me a Coffee Section